INDIAN INSTITUTE OF INFORMATION TECHNOLOGY VADODARA

Department of Computer Applications

SOFTWARE ENGINEERING

PROJECT REPORT

kvstore

A Distributed, Fault-Tolerant, Transactional Key–Value Store
with an Integrated Operational Intelligence Layer

Consolidated Product Documentation

Requirements, process model, use cases, data flow, UML design, modular architecture, and functional testing

Programme: Master of Computer Applications (MCA), 3rd Semester
Subject Code: CS545
Academic Session: 2026
Group No.: 1
Team Leader: Neha Nupur – 20251651064
Member: Aditi Garg – 20251651008
Member: Shivam Kumar Soni – 20251651084

Submitted to: Department of Computer Applications
Indian Institute of Information Technology Vadodara
Autumn 2026–27

Certificate

This is to certify that the project documentation titled **"kvstore: A Distributed, Fault-Tolerant, Transactional Key–Value Store with an Integrated Operational Intelligence Layer"** has been prepared by the students listed below as part of the CS545 Software Engineering laboratory work at the Indian Institute of Information Technology Vadodara.

Team Leader	Neha Nupur (20251651064)
Team Members	Aditi Garg (20251651008); Shivam Kumar Soni (20251651084)
Course	MCA, 3rd Semester – Software Engineering (CS545)
Academic Session	2026

Date: _____ Faculty Signature: _____

Faculty Name: _____ Department Seal: _____

Declaration

We declare that this report is a consolidated account of the design and laboratory artefacts prepared for the *kvstore* project. Diagrams supplied with the project materials have been reused where available. Statements about implementation and testing are distinguished from proposed behaviour and test design; no test is described as freshly executed unless a corresponding execution result is available in the supplied evidence.

Neha Nupur	Aditi Garg	Shivam Kumar Soni
20251651064	20251651008	20251651084

Executive Summary

kvstore is a distributed key–value database designed around replicated state, durable logging, transaction isolation, and operational visibility. Its client-facing path accepts key–value operations and transactions, routes requests to the correct shard and Raft leader, replicates proposals, and applies committed entries to storage. The storage design includes a write-ahead log (WAL), hybrid logical clock (HLC) timestamps, and multi-version concurrency control (MVCC). Operational interfaces expose metrics and an AI-assisted diagnostic path that can submit bounded anomaly evidence to an external large-language-model provider and receive root-cause hypotheses.

This report unifies the work represented by the Software Requirements Specification (Lab 1), process-model selection (Lab 2), use-case modelling (Lab 3), data-flow modelling (Lab 4), object-oriented and interaction design artefacts (Tutorials 5 and 6), and modular design and functional test design (Labs 8 and 9). It is written as product documentation: it explains what the system is for, how its components collaborate, what requirements are testable, and what evidence is available for verification.

Evidence convention. A requirement or test case in this report describes intended or specified behaviour. A test marked as covered by source tests is not automatically a claim that the test was rerun for this report. Fresh execution status should only be recorded from a reproducible run against a named build.

Contents

1 Introduction and Product Overview **1**

1.1 Purpose . . . 1

1.2 Problem Statement . . . 1

1.3 Goals . . . 1

1.4 Scope and Boundaries . . . 1

1.4.1 In scope . . . 1

1.4.2 Out of scope or externally dependent . . . 2

1.5 Stakeholders . . . 2

2 System Context and High-Level Architecture **3**

2.1 External Entities . . . 3

2.2 Context and Data-Flow Views . . . 3

2.3 Subsystem Overview . . . 4

2.4 Internal Package Architecture . . . 4

3 Requirements Specification **5**

3.1 Functional Requirements . . . 5

3.2 Non-Functional Requirements . . . 10

3.3 Constraints and Assumptions . . . 11

4 Software Process Model and Development Plan **12**

4.1 Selection Rationale . . . 12

4.2 Chosen Approach: Risk-Driven Iterative Incremental Development . . . 12

4.3 Risk Register . . . 12

4.4 Case Analyses from the Lab 2 Handout . . . 13

4.4.1 Case 1: College Payroll System — Waterfall . . . 13

4.4.2 Case 2: New Food-Delivery App — Agile . . . 13

4.4.3 Case 3: Hospital Emergency Monitoring System — Spiral . . . 13

4.4.4 Case 4: New College Portal Interface — Prototyping . . . 14

4.5 Decision Factors and Weighted Decision Matrix . . . 14

4.6 Practical Process Controls . . . 14

4.7 Development Increments . . . 15

5 Use-Case Model **16**

5.1 Use-Case Diagram . . . 16
5.2 Actors . . . 16
5.3 Principal Use-Case Descriptions . . . 17
5.4 Use-Case Relationships . . . 17

6 Data-Flow Modelling **19**
6.1 Level 0: Context Diagram . . . 19
6.2 Level 1: Subsystem Data Flow . . . 21
6.3 Level 2: Internal Package Architecture . . . 23
6.4 Level 2 Internal Package Interpretation . . . 24
6.4.1 Named internal flows . . . 24
6.5 Balancing and Consistency Check . . . 25

7 UML Structural Design **26**
7.1 Class Diagram . . . 26
7.2 Important Classes and Responsibilities . . . 28
7.3 Design Principles Reflected . . . 28

8 Interaction and Sequence Design **29**
8.1 Scenario 1: PUT with Leader Redirect . . . 29
8.2 Scenario 2: Transaction with Snapshot Isolation . . . 30
8.3 Scenario 3: Cross-Shard Two-Phase Commit . . . 31
8.4 Cross-Scenario Invariants . . . 32

9 Modular Design **33**
9.1 Design Objectives . . . 33
9.2 Module Architecture . . . 33
9.3 Module Responsibilities and Interfaces . . . 33
9.4 Cohesion and Coupling Review . . . 34
9.5 Architecture Decisions . . . 35

10 Implementation Overview **36**
10.1 Technology and Repository View . . . 36
10.2 Key Implementation Areas . . . 36
10.3 Error Handling Principles . . . 36

11 Functional Test Strategy and Verification **38**
11.1 Testing Objectives . . . 38
11.2 Test Levels . . . 38
11.3 Test Register . . . 38
11.4 Detailed Test Procedure: PUT and GET . . . 39
11.5 Detailed Test Procedure: MVCC Conflict . . . 40
11.6 Detailed Test Procedure: Crash and Restart Recovery . . . 40

11.7 Detailed Test Procedure: Cross-Shard Atomicity . . . 40
11.8 Traceability Summary . . . 41
11.9 Implementation Evidence and Status Discipline . . . 41

12 Security, Reliability, and Operational Considerations 42
12.1 Security . . . 42
12.2 Reliability and Recovery . . . 42
12.3 Operational Monitoring . . . 42

13 Limitations and Future Enhancements 43
13.1 Current Documentation Limitations . . . 43
13.2 Recommended Enhancements . . . 43

14 Conclusion 44

A Consolidation Map: Laboratory Artefacts 45

B Expanded Lab 8 Modular-Design Record 46
B.1 Project Requirements Used for Modularisation . . . 46
B.2 Responsibility Inventory . . . 46
B.3 Thirteen Modules and Package Mapping . . . 47
B.4 Module Interfaces . . . 47
B.4.1 Interface 1: Transaction API . . . 47
B.4.2 Interface 2: Wire Envelope . . . 48
B.4.3 Interface 3: StatsProvider . . . 48
B.4.4 Interface 4: Key Routing . . . 48
B.5 Cohesion and Coupling Decisions . . . 49
B.6 Coupling Numericals from Lab 8 . . . 49
B.7 Module-to-Code Traceability from Lab 8 . . . 50

C Expanded Lab 9 Test Register and Execution Detail 51
C.1 Selected Lab 9 Requirements . . . 51
C.2 Original Register A1: Scenario and Classification . . . 52
C.3 Test Procedure Details and Pass Rules . . . 53
C.3.1 TC-01: Create a key and read it back . . . 53
C.3.2 TC-06: Conflicting MVCC writes . . . 53
C.3.3 TC-07: Crash and restart recovery . . . 53
C.3.4 TC-08: Request through a follower . . . 53
C.3.5 TC-10: Prometheus metrics . . . 53
C.3.6 TC-11: Unsupported operational query . . . 54
C.3.7 TC-12: Untrusted TLS certificate . . . 54
C.3.8 TC-14 and TC-15: Cross-shard transaction boundary . . . 54
C.4 Repository Test Evidence Named in Lab 9 . . . 54

C.5 Traceability and Execution Record 54

D Quick Reference: Key Terms **56**

E Team and Submission Information **57**

List of Figures

5.1 Supplied use-case diagram for the distributed transactional key–value store. 16

6.1 Level 0 context diagram. Original supplied image retained; shown separately from Level 1 and Level 2. 19
6.2 Level 1 subsystem DFD. Original supplied image retained as a standalone portrait-page figure. 21
6.3 Level 2 internal package architecture. Original supplied image retained as a separate portrait-page figure. 23

7.1 UML class diagram supplied for kvstore. 27

8.1 Sequence diagram for a PUT request when the contacted node may not be the leader. 29
8.2 Sequence diagram for a transaction using snapshot visibility and first-committer-wins validation. 30
8.3 Sequence diagram for a transaction spanning two shards using two-phase commit. . . 31

9.1 Module architecture for kvstore, based on the supplied Lab 8 artefact. 33

List of Tables

4.2 Weighted process-model comparison reproduced from the Lab 2 analysis. 14

Chapter 1

Introduction and Product Overview

1.1 Purpose

The purpose of *kvstore* is to provide a reliable key–value service that can continue to operate in a distributed cluster while maintaining a consistent committed state. In addition to core data operations, the system includes administrative observability to help an operator understand cluster behaviour and investigate anomalies.

1.2 Problem Statement

A single-process in-memory key–value map is easy to implement but does not by itself provide replicated durability, leadership handling, transaction isolation, recovery after a crash, shard-aware routing, or operational diagnosis. *kvstore* addresses these concerns through distinct subsystems and explicit protocols. The system must also make failures visible through clear API outcomes rather than silently applying partial or conflicting writes.

1.3 Goals

- Provide create/update, retrieve, and delete operations for key–value entries.
- Route writes through the correct shard and active Raft leader.
- Replicate log entries and commit them only when the required consensus conditions are met.
- Preserve committed writes through durable logging and restart recovery.
- Support transactions with snapshot visibility and conflict detection.
- Coordinate multi-shard transactions with a durable two-phase-commit decision record.
- Expose metrics and operational diagnostics while keeping the AI provider outside the data-commit trust boundary.
- Validate operational endpoints, TLS trust, and restricted natural-language queries.

1.4 Scope and Boundaries

1.4.1 In scope

Core key–value operations; shard routing; Raft leader and follower behaviour; WAL-backed recovery; MVCC and HLC; transaction commit/abort; cross-shard coordination; REST/JSON interfaces; metrics; TLS validation; and bounded AI-assisted operational queries.

1.4.2 Out of scope or externally dependent

The external LLM is not a database authority and must not decide whether a transaction commits. Its output is advisory. Production deployment topology, authentication provider configuration, performance service-level objectives, and operational retention settings depend on the selected deployment and must be confirmed before production use.

1.5 Stakeholders

Stakeholder	Interest / responsibility
Client application	Submits key–value and transaction requests; consumes results and errors.
Cluster operator / administrator	Monitors health, reviews metrics and alerts, manages nodes and certificates, and investigates incidents.
System administrator	Configures deployment, security, and operational policy.
External LLM provider	Receives bounded diagnostic evidence and returns hypotheses; it is not trusted with commit authority.
Developer / tester	Maintains modules, validates requirements, and records repeatable test evidence.

System Context and High-Level Architecture

2.1 External Entities

Entity	Input to kvstore	Output from kvstore
Client application	PUT/GET/DELETE operations, transaction commands, request metadata	Values, acknowledgements, transaction outcomes, redirects, and error responses
Cluster operator	Dashboard and metrics queries; operational questions; cluster-management actions	Live dashboard, performance measures, alerts, diagnostic insights, and configuration outcomes
External LLM provider	Receives anomaly evidence / diagnostic context from the observability layer	Returns root-cause hypotheses or explanations

2.2 Context and Data-Flow Views

The context, Level 1, and Level 2 diagrams are presented together in the dedicated Data-Flow Modelling chapter, each as a separate portrait page. This separation is intentional: it preserves the original diagram images and labels, avoids stacking different abstraction levels into one figure, and gives each diagram its own interpretation and data-flow inventory. Figure 6.1 defines the external boundary; Figure 6.2 decomposes the system into major processes; Figure 6.3 expands those processes into internal packages.

2.3 Subsystem Overview

Subsystem	Primary responsibility	Principal information handled
1.0 API & Routing	Accepts requests, validates input, routes by key/shard, and handles leader redirects.	Requests, route information, client outcomes
2.0 Consensus	Uses Raft to elect leaders, replicate proposals, and determine commitment.	Proposals, acknowledgements, committed entries
3.0 Storage	Applies committed entries and manages durable state, HLC, and MVCC.	Committed entries, versions, WAL records
4.0 Observability	Collects performance signals, serves metrics, and produces diagnostic insights.	Metric streams, alerts, anomaly evidence

2.4 Internal Package Architecture

The Level 2 view expands the subsystems into packages: HTTP API/transport, client library, shard routing, Raft, WAL, storage, HLC, MVCC, metrics, and the operational intelligence engine. The full supplied diagram and its package-level interpretation appear in Chapter 6. The package view makes responsibilities explicit and supports testing individual modules as well as end-to-end flows.

Chapter 3

Requirements Specification

3.1 Functional Requirements

The formal SRS defines requirements using identifiers `SRS FR 001` through `SRS FR 084`. The table below preserves that identifier namespace and summarises each requirement from the supplied SRS, grouped in the original requirement order. This is the authoritative requirements index for this consolidated report.

SRS ID	Requirement summary from the SRS
SRS FR 001	The system shall generate, for every event requiring a timestamp, a composite value consisting of a physical wall clock reading, a logical counter, and the identifier of the generating node.
SRS FR 002	The system shall increase the logical counter component of a generated timestamp whenever the physical wall clock reading has not advanced relative to the previous timestamp generated by the same node.
SRS FR 003	The system shall provide a comparison function over generated timestamps that orders first by physical component, second by logical component, and third by node identifier, yielding a strict total order across the entire cluster.
SRS FR 004	The system shall provide a mechanism for folding a timestamp received from a remote node into the local clock state such that causal ordering between the local node and the remote node is preserved when that mechanism is invoked.
SRS FR 005	The system shall support a configurable maximum tolerated clock drift and shall support the artificial injection of a clock offset for the purpose of automated testing of clock skew handling.
SRS FR 006	The system shall document, as a known and explicit limitation rather than as an implicit assumption, that no message currently transmitted over the network protocol carries a remote timestamp for automatic folding during ordinary production operation, and that cross node ordering during ordinary operation instead depends on the node identifier component of every timestamp together with each node's physical clock remaining within the configured maximum tolerated drift of its peers.
SRS FR 007	The system shall append an arbitrary byte sequence record to a log file together with a checksum computed over the contents of that record.
SRS FR 008	The system shall force a physical flush of every appended record to the underlying storage device before reporting that record as durably persisted.
SRS FR 009	The system shall, during replay of a log file after a restart, recompute the checksum of every stored record and shall treat the first record whose recomputed checksum does not match its stored checksum, or the first record that is truncated, as the effective end of the log, discarding nothing that precedes it and trusting nothing beyond it.
SRS FR 010	The system shall use this write ahead log mechanism to persist every Raft log entry, the current term and vote record, and every periodic snapshot for each cluster node.

SRS ID	Requirement summary from the SRS
SRS FR 011	The system shall maintain, for every cluster node at every moment, exactly one of the three roles leader, follower, or candidate.
SRS FR 012	The system shall elect at most one leader per Raft term across the entire cluster, verified specifically under contested, near simultaneous election conditions.
SRS FR 013	The system shall use a randomised election timeout, configurable through a minimum and maximum bound, so that followers do not simultaneously become candidates.
SRS FR 014	The system shall require that a candidate's replicated log be at least as up to date as a voting node's own log before that voting node grants its vote, and shall permit a voting node to grant at most one vote per term.
SRS FR 015	The system shall replicate every operation accepted by the leader to every follower and shall consider that operation committed only once a majority of the cluster, including the leader itself, has durably stored it.
SRS FR 016	The system shall detect the point at which a follower's log diverges from the leader's log and shall bring that follower back into agreement using an accelerated conflict back off mechanism rather than retransmitting one entry at a time.
SRS FR 017	The system shall periodically compact its log through snapshotting and shall discard log entries made redundant by a completed snapshot.
SRS FR 018	The system shall transfer a complete snapshot directly to any follower that has fallen so far behind that the leader has already discarded the log entries that follower requires, after which that follower shall resume ordinary incremental replication.
SRS FR 019	The system shall support live changes to cluster membership, including the addition and removal of voting members, using a joint consensus configuration that requires majority agreement from both the old and the new membership lists during the transitional period.
SRS FR 020	The system shall automatically propose the finalising configuration entry, dropping the old membership list, once the joint configuration entry has itself been committed.
SRS FR 021	The system shall support a deliberate, voluntary leadership transfer from the current leader to a specified follower, ensuring the target follower's log is fully caught up before instructing it to begin an election immediately, without waiting for its normal election timeout.
SRS FR 022	The system shall never permit a network partition to allow a minority subset of the cluster to elect a leader or to commit a write, ensuring that a full network partition can never result in the two disjoint sides of the cluster independently and silently diverging into inconsistent data.
SRS FR 023	The system shall store, for every key, an ordered chain of versions, each version tagged with the hybrid logical clock timestamp at which it was written.
SRS FR 024	The system shall assign every new transaction a beginning timestamp representing the instant the transaction started.
SRS FR 025	The system shall ensure that every read performed by a transaction, for the remainder of that transaction's lifetime, returns the version of the requested key that was most recently committed as of that transaction's beginning timestamp, regardless of any transaction that commits later while this transaction is still active.
SRS FR 026	The system shall reject, at commit time, any transaction for which some key it wrote has had a newer version committed since that transaction's beginning timestamp, applying a first committer wins conflict resolution rule.
SRS FR 027	The system shall perform the authoritative conflict check for a committing transaction deterministically, at the moment that transaction's write set is applied from the Raft log on every replica, so that every replica independently reaches the identical commit or abort decision.

SRS ID	Requirement summary from the SRS
SRS FR 028	The system shall additionally perform an optimistic, non authoritative conflict check before a transaction is proposed through Raft at all, so that a transaction that is already certain to fail can be rejected quickly without incurring the cost of a full consensus round trip.
SRS FR 029	The system shall assign every transaction a unique identifier derived deterministically from that transaction's hybrid logical clock beginning timestamp tuple, ensuring uniqueness across the whole cluster and across process restarts.
SRS FR 030	The system shall reclaim, through a garbage collection process, any version of a key older than the beginning timestamp of the oldest still active transaction in the cluster, while never reclaiming the single most recent remaining version of any key.
SRS FR 031	The system shall support a temporary, exclusive intent lock on a set of keys on behalf of a transaction that has been tentatively prepared but not yet resolved, during which no ordinary read is permitted to observe the locked keys in an uncommitted state.
SRS FR 032	The system shall expose a transactional application programming interface supporting the operations begin, get, put, delete, and commit, callable by code elsewhere in the system.
SRS FR 033	The system shall, upon a commit request, propose the transaction's complete write set through Raft as a single log entry and shall wait for that entry to be safely and deterministically applied to a majority of the cluster before reporting success to the caller.
SRS FR 034	The system shall function as the Raft state machine, meaning it shall receive every committed log entry from Raft in order, on every node, and shall interpret and apply that entry to the actual key value data.
SRS FR 035	The system shall encode every request as a single structured envelope message using a built in binary serialisation format and shall transmit it over a freshly opened connection for each individual request.
SRS FR 036	The system shall dispatch both internal Raft protocol messages and external client key value requests through the same server process and the same listening network address.
SRS FR 037	The system shall, when a client request arrives at a node that is not the current leader, respond with a specific reply indicating that condition, and shall include, whenever known, the network address of the current leader as a hint.
SRS FR 038	The system shall support an entirely optional Transport Layer Security encrypted mode for every connection, activated additively without changing default behaviour when disabled.
SRS FR 039	The system shall support an optional mutual Transport Layer Security mode requiring every connecting client to present a certificate signed by a configured trusted authority.
SRS FR 040	The system shall provide a client library capable of discovering which cluster node is currently the leader.
SRS FR 041	The system shall automatically redirect a client request to the correct leader node when an initial request is answered with a not leader reply carrying a leader hint.
SRS FR 042	The system shall retry a client request across multiple candidate cluster addresses when one or more addresses are temporarily unreachable, rather than failing immediately upon the first unreachable address.
SRS FR 043	The system shall provide monotonically increasing counters, freely adjustable gauges, and histograms as its three fundamental instrumentation primitives.
SRS FR 044	The system shall compute, for every histogram, the count, sum, minimum, maximum, running mean, and running standard deviation of recorded observations using a numerically stable, single pass algorithm.
SRS FR 045	The system shall compute an exponentially weighted moving average for every histogram, emphasising the most recent observations over older ones.

SRS ID	Requirement summary from the SRS
SRS FR 046	The system shall compute approximate percentile estimates for every histogram using a fixed set of logarithmically sized buckets.
SRS FR 047	The system shall automatically measure the latency and success or failure outcome of every client request handled by the network server, broken down by the specific operation performed.
SRS FR 048	The system shall expose accumulated metrics data in both the standard Prometheus text exposition format and a plain JSON format.
SRS FR 049	The system shall detect anomalies in a monitored metric series using two exponentially weighted moving averages tracked in parallel, one representing a slowly moving historical baseline and one representing rapidly moving current behaviour, and shall flag an anomaly when the current estimate diverges sufficiently from the baseline relative to that metric's own recent variability.
SRS FR 050	The system shall compute the baseline variance estimate used in anomaly detection using a mechanism that is robust against being dragged upward by a sustained anomaly, so that a persistent problem does not gradually retrain the baseline to consider that problem normal.
SRS FR 051	The system shall retain, for every tracked metric, its recent raw observation history in a bounded, fixed size rolling buffer, distinct from that metric's smoothed moving average values.
SRS FR 052	The system shall, upon the detection of a critical anomaly in one metric, automatically compute the Pearson correlation coefficient between that metric and every other tracked metric over the same recent time window, and shall report the resulting correlated series together with the numerical strength of each correlation.
SRS FR 053	The system shall fit a least squares regression line through a tracked metric's recent raw history and shall project that line forward to produce a trend forecast, reporting alongside that forecast a coefficient of determination indicating the goodness of fit of the underlying trend line.
SRS FR 054	The system shall continuously score every node in the cluster on a combination of recently observed latency, error rate, replication lag, and queue depth.
SRS FR 055	The system shall recommend a leadership transfer away from the current leader when a follower is clearly and consistently healthier by the node scoring mechanism, applying a hysteresis margin to prevent the recommendation from oscillating between two nodes of similar health.
SRS FR 056	The system shall never allow the leadership advisor to directly instruct the consensus layer to perform a transfer; the advisor shall only ever produce a recommendation, with any actual transfer requiring either an explicit operator request or an explicit, separately configured automatic transfer setting.
SRS FR 057	The system shall adaptively adjust the garbage collection interval and the automatic snapshotting threshold using a proportional feedback controller driven by the currently observed write rate and the degree of accumulated multi version concurrency control version bloat.
SRS FR 058	The system shall answer common, plainly phrased natural language questions about cluster state, including questions about the number of stored keys, the identity of the current leader, recently detected anomalies, and a forecast for a named metric, using a fully deterministic, rule based parser requiring no external network dependency.
SRS FR 059	The system shall, only when an external large language model credential has been explicitly configured by the operator, fall back to a large language model for natural language questions the deterministic parser does not recognise, and shall constrain that large language model to selecting exclusively from among the same fixed, pre validated set of read only operations the deterministic parser itself supports, independently verifying every selection against that fixed list before executing it.

SRS ID	Requirement summary from the SRS
SRS FR 060	The system shall, only when an external large language model credential has been explicitly configured, optionally hand already computed anomaly and correlation evidence to a large language model to obtain a plain language, prose only root cause hypothesis and suggested next step, and shall ensure this diagnosis capability never returns anything that is executed, runs asynchronously under a rate limit, and never blocks the system's own continuous monitoring loop.
SRS FR 061	The system shall degrade gracefully and silently whenever the optional large language model call fails or is not configured, such that the deterministic anomaly and correlation insights already computed and recorded remain fully valid and available regardless.
SRS FR 062	The system shall implement the entire operational intelligence layer without any direct dependency on the consensus, storage, or network transport code, obtaining all required cluster data exclusively through two narrowly defined interfaces supplied by an external adapter.
SRS FR 063	The system shall generate a temporary, self signed certificate suitable for local development use when Transport Layer Security is enabled without an explicitly supplied certificate and key.
SRS FR 064	The system shall load a real certificate and private key pair supplied by the operator for production deployment when explicitly configured to do so.
SRS FR 065	The system shall support requiring every connecting client to present a certificate signed by a configured trusted authority when mutual Transport Layer Security is enabled.
SRS FR 066	The system shall expose, on every node for which the Hypertext Transfer Protocol interface is enabled, a REST style application programming interface supporting reading, writing, and deleting a single key, submitting a multi key transaction, browsing the locally visible set of keys, retrieving node status, retrieving metrics in either supported format, and interacting with every capability of the operational intelligence layer.
SRS FR 067	The system shall expose a continuous, live streaming feed of cluster events using the Server Sent Events web standard, chosen specifically because the required data flow is one directional, from server to browser, and because the standard provides automatic client side reconnection without additional implementation effort.
SRS FR 068	The system shall serve a single, self contained, embedded browser dashboard requiring no separately deployed files and no externally fetched resources, presenting a live visual representation of the cluster arranged as nodes in a circle with visible connections to the current leader that pulse on each heartbeat, live charts of latency, throughput, and error rate, a browser for currently stored keys, a feed of operational intelligence insights, and a text input for submitting natural language questions.
SRS FR 069	The system shall allow every node to optionally poll the status endpoint of every other configured peer node over ordinary Hypertext Transfer Protocol, building a complete, honestly self reported picture of cluster wide health for the leadership advisor, and shall ensure that a node with no peer polling configured limits itself to what it can observe locally and never recommends a leadership transfer in that configuration.
SRS FR 070	The system shall route each incoming key to exactly one of a configured set of independent shard clusters using rendezvous hashing, computing a deterministic numeric score for the key against every configured shard using the SHA two hundred fifty six hash function and selecting the shard with the highest score.
SRS FR 071	The system shall ensure that the routing decision for any given key depends only on the fixed, configured set of shard identifiers and on the key itself, and never on any per process or per run randomised state, so that the identical routing decision is reached by any process constructing the routing table over the same shard set, including the same process across a restart.

SRS ID	Requirement summary from the SRS
SRS FR 072	The system shall reject, before forwarding it to any shard, any ordinary multi key transaction whose keys span more than one shard, returning a clear error to the caller.
SRS FR 073	The system shall provide a dedicated cross shard transaction endpoint supporting atomic, multi key writes and deletes across as many shards as the affected keys are routed to, implemented through a two phase commit coordinator.
SRS FR 074	The system shall, as the first step of any cross shard transaction, durably record which shards that transaction will involve before contacting any of them.
SRS FR 075	The system shall, during the prepare phase of a cross shard transaction, request that every involved shard lock its affected keys using the intent mechanism, aborting the entire transaction if any single shard declines to prepare.
SRS FR 076	The system shall, once every involved shard has successfully agreed to prepare, durably record the final commit decision before instructing any shard to act upon it, this durable recording being the exact point at which the transaction's outcome becomes permanently fixed.
SRS FR 077	The system shall instruct every involved shard to carry out the recorded commit or abort decision, retrying past transient failures until every shard has confirmed completion.
SRS FR 078	The system shall, upon restart after a crash occurring during a cross shard transaction, replay its own durably recorded decision log and resolve any transaction for which no final decision was durably recorded before the crash as an abort.
SRS FR 079	The system shall decline, with a clear explanatory error, any cross shard transaction request received by a gateway instance that was not configured with its own durable data directory.
SRS FR 080	The system shall provide a server program, one running instance of which constitutes exactly one member of a Raft cluster, configurable through command line flags governing network addresses, data directory location, consensus timing, optional Transport Layer Security, and optional operational intelligence settings.
SRS FR 081	The system shall provide a command line client program supporting reading, writing, deleting a key, and checking a node's status, built directly upon the shared client library.
SRS FR 082	The system shall provide a sharding gateway program configured with the network addresses of each independent shard cluster it routes to, which shall function correctly for ordinary routed operations even without a configured data directory, declining only cross shard transaction requests in that configuration.
SRS FR 083	The system shall detect, through a stored per record checksum, any corruption of a write ahead log record caused by a partial write during a crash, and shall treat the log as ending at the first such corrupted or truncated record during replay.
SRS FR 084	The system shall ensure that every replica of the cluster applies the identical sequence of Raft log entries in the identical order, which is the property that guarantees every replica's multi version concurrency control state remains consistent with every other replica's state.

3.2 Non-Functional Requirements

Category	Expectation
Consistency	Replicas should converge on the same committed log and deterministic state transitions.
Durability	Committed writes must survive a process crash when the configured durable storage guarantees hold.

Category	Expectation
Availability	The cluster should continue to accept eligible operations when a functioning leader and required quorum are available.
Security	TLS trust must be validated; administrative and operational surfaces must be constrained.
Maintainability	Modules should have high cohesion, low coupling, and explicit interfaces.
Observability	Metrics and diagnostic events should help operators understand latency, errors, and cluster state.
Recoverability	Startup recovery must reconstruct committed state and resolve durable transaction decisions.
Testability	Requirements should map to deterministic test procedures and observable expected outcomes.

3.3 Constraints and Assumptions

- Raft correctness depends on correct quorum, leader, and log-commit behaviour.
- A local test run does not by itself prove behaviour under production network partitions or hardware failure.
- TLS tests must distinguish secure defaults from explicitly configured insecure test options.
- LLM explanations can be incomplete or incorrect and must not be treated as authoritative database state.
- Throughput, latency targets, cluster size limits, and deployment-specific retention policies require explicit operational values before release.

Chapter 4

Software Process Model and Development Plan

4.1 Selection Rationale

The Lab 2 artefact evaluates process-model selection for a project with distributed protocols, transaction semantics, recovery requirements, security concerns, and evolving observability features. A purely linear process is less suitable because protocol assumptions and integration risks need repeated validation. A risk-driven, iterative and incremental approach is a good fit: each increment delivers a coherent capability while high-risk behaviours are prototyped and tested early.

4.2 Chosen Approach: Risk-Driven Iterative Incremental Development

1. **Requirements baseline:** identify core API, replication, durability, transaction, security, and observability requirements.
2. **Architecture and risk analysis:** define subsystem boundaries and assess risks such as stale leader routing, WAL recovery, MVCC conflicts, and cross-shard in-doubt outcomes.
3. **Incremental implementation:** build and validate transport/routing, consensus, storage, transactions, and observability in controlled increments.
4. **Integration and regression:** run unit, package, integration, and end-to-end tests at each increment.
5. **Review and refinement:** update requirements, diagrams, interfaces, and test traceability when design decisions change.

4.3 Risk Register

ID	Risk	Impact	Mitigation / evidence
R1	Incorrect leader routing	Writes may be rejected or sent to a follower	Leader hint, redirect-and-retry tests, leader-change integration tests

ID	Risk	Impact	Mitigation / evidence
R2	WAL durability gap	Committed data may be lost after restart	Durable append before commitment acknowledgement; crash/restart recovery test
R3	MVCC conflict bug	Lost update or inconsistent transaction result	Concurrent-write tests and first-committer-wins assertions
R4	Cross-shard partial commit	Atomicity violation across shards	Durable decision log, prepare/commit/abort protocol, crash-recovery tests
R5	Unsafe operational query	Unexpected or privileged operation	Restricted command grammar, safe fallback, negative tests
R6	Untrusted TLS endpoint	Connection security can be bypassed	Certificate-chain validation and negative trust test
R7	LLM provider outage or incorrect answer	Misleading operational advice	Keep LLM advisory, bound diagnostic context, handle provider failure without affecting commit path

4.4 Case Analyses from the Lab 2 Handout

The process-model exercise compares the fit of five models against requirement stability, early delivery, feedback, risk handling, and testing. The four classroom cases below are retained from the original assignment because they demonstrate why model selection depends on the problem rather than on a universal ranking.

4.4.1 Case 1: College Payroll System — Waterfall

Salary rules, allowances, and deductions are known and major changes are not expected. Requirements are stable and documentable; payroll/HR can review calculations at planned checkpoints. Waterfall is suitable because requirements can be approved up front and calculation rules can be traced through requirements, design, implementation, testing, and deployment. Payroll accuracy still requires thorough validation.

4.4.2 Case 2: New Food-Delivery App — Agile

The startup expects frequent releases and users may request features after trying the application. Ordering, payment, delivery tracking, and restaurant workflows benefit from a minimum viable feature set and continuous feedback. Agile short iterations allow the team to reprioritise a backlog; continuous integration and regression testing protect existing flows.

4.4.3 Case 3: Hospital Emergency Monitoring System — Spiral

The consequences of failure are serious, so technical and safety risks must be studied explicitly. Spiral fits because each cycle defines objectives and alternatives, analyses risks, builds and verifies a solution,

and plans the next cycle. A real medical system would additionally require applicable regulatory, clinical, and safety-engineering processes.

4.4.4 Case 4: New College Portal Interface — Prototyping

Students and staff are uncertain about screens and want samples before finalising requirements. A low-cost interactive mock-up gives stakeholders something concrete to evaluate for navigation, terminology, accessibility, and task completion. The prototype is exploratory unless it is engineered to production quality.

4.5 Decision Factors and Weighted Decision Matrix

The original Lab 2 analysis identifies the following kvstore characteristics: core operations are clear but edge cases, operational intelligence, sharding, and recovery require refinement; technical complexity is high because Raft, MVCC, WAL, HLC, TLS, network transport, and cross-shard coordination interact; failure impact is high because lost or inconsistent data undermines the system; early verification is essential; feedback comes from tests, fault injection, metrics, dashboard behaviour, and review; and the modular Go architecture supports incremental implementation.

Ratings are qualitative judgements, not measurements. Each criterion is rated from 1 (weak fit) to 5 (strong fit), then multiplied by its weight.

Criterion	Weight	Waterfall	Incremental	Prototype	Spiral	Agile
Requirement evolution	15%	4	4	5	4	5
Risk analysis	30%	2	3	2	5	3
Early verification	20%	2	4	4	4	5
Architectural complexity	20%	2	4	2	5	3
Feedback / adaptation	15%	2	3	5	4	5
Weighted score / 5	100%	2.40	3.60	3.25	4.50	4.00

Table 4.2: Weighted process-model comparison reproduced from the Lab 2 analysis.

The calculation is score = $\sum$(criterion weight $\times$ rating). Spiral receives the highest score (4.50/5) because risk analysis has the greatest weight. The original recommendation is therefore **Spiral as the primary model, with incremental and Agile-style delivery practices**. This is a recommendation based on the project's risk profile, not a claim that the repository historically declared a formal lifecycle.

4.6 Practical Process Controls

The lab defines a repeatable cycle: plan a small testable objective and identify affected SRS requirements; analyse failure modes, impact, likelihood, and mitigation; design interfaces while preserving the modular dependency hierarchy; implement one vertical slice or module increment; verify with unit/integration tests, fault injection, crash-recovery tests, race detection, build and vet checks as appropriate; review test evidence, metrics, defects and limitations; then re-plan according to the evidence. An untested capability must not be treated as verified.

4.7 Development Increments

Increment	Deliverable	Exit evidence
Foundation	API, client and shard routing	Valid request/response; invalid request handling; routing tests
Replication	Raft log, WAL and durable commit	Leader/follower tests; replication acknowledgements; restart recovery
Transactions	HLC, MVCC and transaction API	Snapshot read tests; conflict/abort tests
Distributed transactions	Shard gateway and two-phase commit	Prepare/commit/abort; atomicity and coordinator-recovery tests
Operations	Metrics, dashboard and operational query	Metrics endpoint; safe query fallback; TLS checks
AI-assisted diagnostics	Anomaly evidence and LLM hypotheses	Failure isolation; no effect on database commit path

Chapter 5

Use-Case Model

5.1 Use-Case Diagram

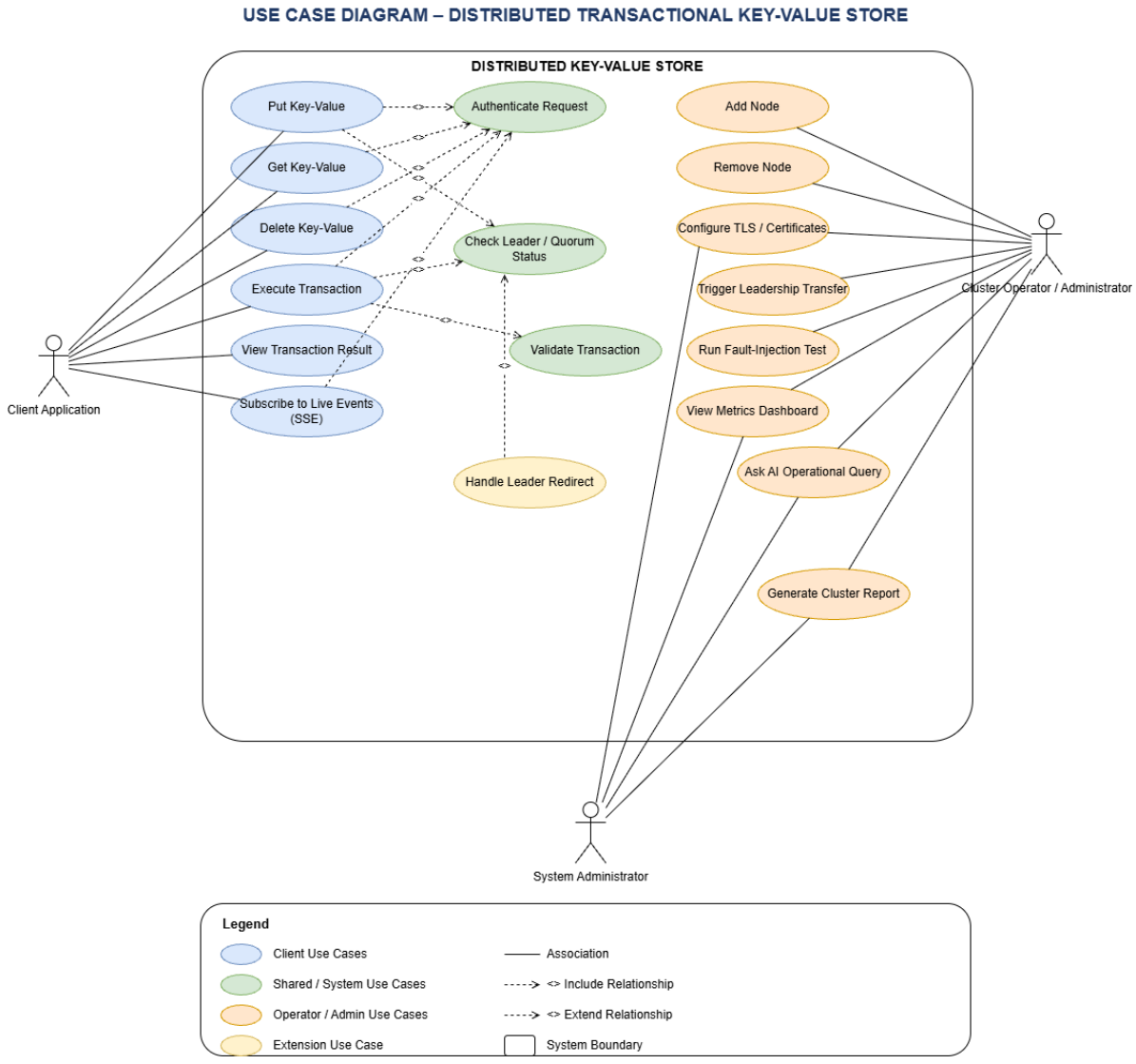


Figure 5.1: Supplied use-case diagram for the distributed transactional key–value store.

5.2 Actors

- **Client Application:** performs PUT, GET, DELETE, transaction execution, reads transaction results, and may subscribe to live events.

- **Cluster Operator / Administrator:** manages membership, TLS/certificates, leadership, fault-injection tests, metrics, AI operational queries, and reports.
- **System Administrator:** performs broader system configuration and privileged administration.
- **External LLM Provider:** an external service used by the observability/diagnostic flow; it is not a primary actor in data mutation.

5.3 Principal Use-Case Descriptions

Use case	Main flow	Alternative / failure flow
Put key–value	Client sends key and value; system routes to shard/leader; entry is replicated and committed; success is returned.	Follower redirects to leader; validation or quorum failure returns a controlled error.
Get key–value	Client sends key; shard returns the visible value.	Absent key returns not-found; transient routing issue may trigger leader discovery.
Delete key–value	Delete is routed and committed; later reads reflect the deletion semantics.	Absent key follows the API's documented idempotency/not-found policy.
Execute transaction	Begin timestamp is assigned; reads use the snapshot; writes are buffered; commit validates conflicts.	Conflicting transaction is aborted; no partial conflicting write is applied.
Cross-shard transaction	Gateway identifies shards; coordinator durably records decision; participants prepare; coordinator commits or aborts.	A failed participant causes abort; restart recovery consults durable decision state.
View metrics dashboard	Operator queries metrics and receives performance/health information.	Unavailable metrics are surfaced as unavailable rather than fabricated.
Ask AI operational query	Supported operational intent is parsed; relevant evidence is assembled; diagnostic hypothesis is returned.	Unknown/unsafe command falls back to help or unsupported-command response.
Configure TLS	Administrator supplies certificate configuration; system validates trust.	Untrusted certificate is rejected in secure mode.

5.4 Use-Case Relationships

The supplied diagram uses include relationships for common supporting behaviour such as request authentication and validation, and extension relationships for optional or conditional flows such as leader redirect. The diagram should be read with the system boundary in mind: client operations are

separated from operator-only actions, while shared validation and routing concerns are reused across multiple use cases.

Data-Flow Modelling

6.1 Level 0: Context Diagram

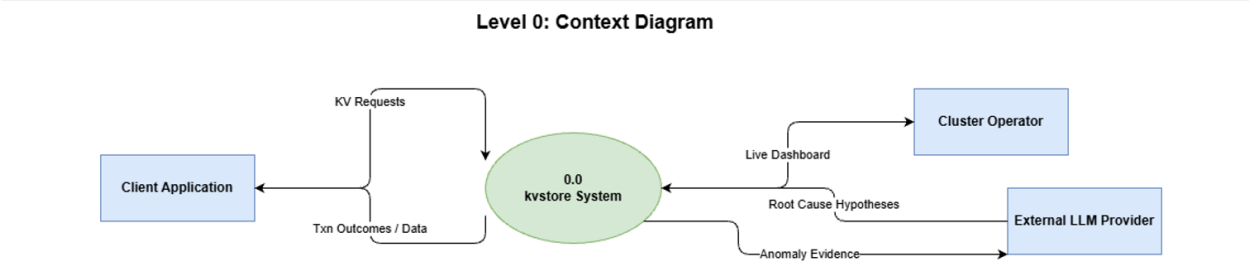


Figure 6.1: Level 0 context diagram. Original supplied image retained; shown separately from Level 1 and Level 2.

At context level, the complete system is one process, labelled 0.0 *kvstore System*. The diagram is a boundary view rather than a deployment topology. It names three external entities and the information exchanged with each:

External entity	**Flow label in supplied diagram**	**Meaning**
Client Application	KV Requests	Requests for key–value operations and transactions enter the system.
kvstore System → Client Application	Txn Outcomes / Data	The result, retrieved data, or transaction outcome returns to the calling client.
Cluster Operator	Live Dashboard	Operational and cluster information is presented to the operator.
kvstore System → External LLM Provider	Anomaly Evidence	Selected operational evidence is sent for diagnostic interpretation.
External LLM Provider → kvstore System	Root Cause Hypotheses	The provider returns hypotheses that may assist an operator; these are advisory and do not control commit decisions.

The system boundary is important: the client and operator interact with *kvstore*, while the external LLM is a separate provider. The diagram does not grant the LLM authority to mutate data or determine consensus. Its two flows are diagnostic evidence out and hypotheses back. The client-facing data flow remains separate from this optional diagnostic interaction.

6.2 Level 1: Subsystem Data Flow

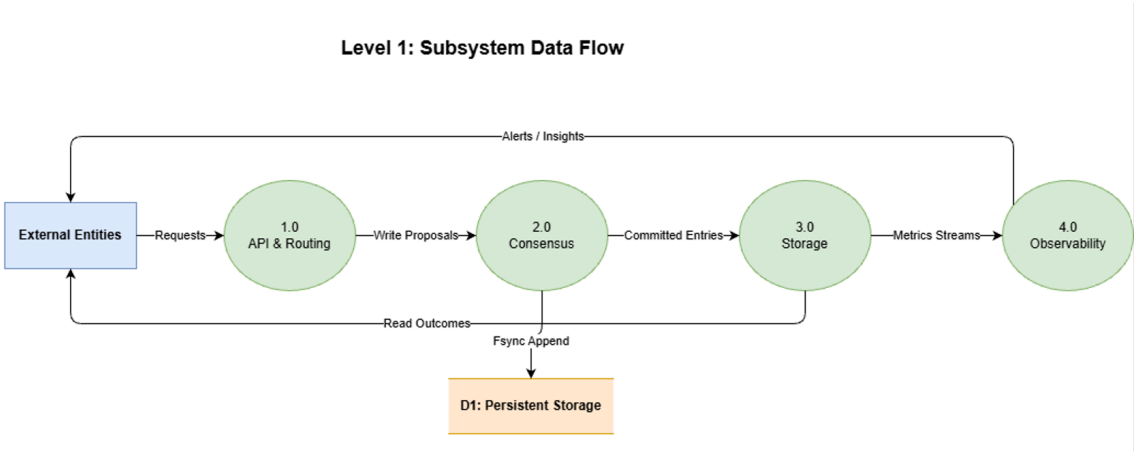


Figure 6.2: Level 1 subsystem DFD. Original supplied image retained as a standalone portrait-page figure.

The Level 1 diagram decomposes the single context process into four numbered processes. The incoming and outgoing flows should be read as labelled in the original artefact, rather than inferred from physical placement alone.

Process / store	**Named flow(s)**	**Interpretation**
External Entities	Requests; Read Outcomes; Alerts / Insights	Represents the external client/operator boundary. Requests enter API and routing; outcomes return from the system; observability returns alerts and insights to the external side.
1.0 API & Routing	Requests; Write Proposals	Accepts requests and forwards proposed state changes toward consensus. This process is also the boundary where routing and leader redirection are handled.
2.0 Consensus	Write Proposals; Committed Entries; Fsync Append	Receives proposed operations, coordinates Raft replication and commitment, and relies on durable log append before treating records as safely persisted.
3.0 Storage	Committed Entries; Metrics Streams; Read Outcomes	Applies committed entries to state, maintains the stored key/value and version state, serves read outcomes, and contributes performance/operation signals.
4.0 Observability	Metrics Streams; Alerts / Insights	Consumes measurements and produces operator-facing insights and alerts. The AI/diagnostic path is an adjunct to operations, not the commit authority.
D1: Persistent Storage	Fsync Append	Durable data written by the log path supports recovery after process restart.

The principal processing path is *External request* → *API & Routing* → *Consensus* → *Storage*. The labelled *Committed Entries* flow indicates that storage applies committed operations rather than treating an uncommitted proposal as final. The *Fsync Append* flow points to D1, highlighting the durability boundary. Read outcomes return to the external side, while metrics travel from storage toward observability and alerts/insights travel back to the external boundary.

6.3 Level 2: Internal Package Architecture

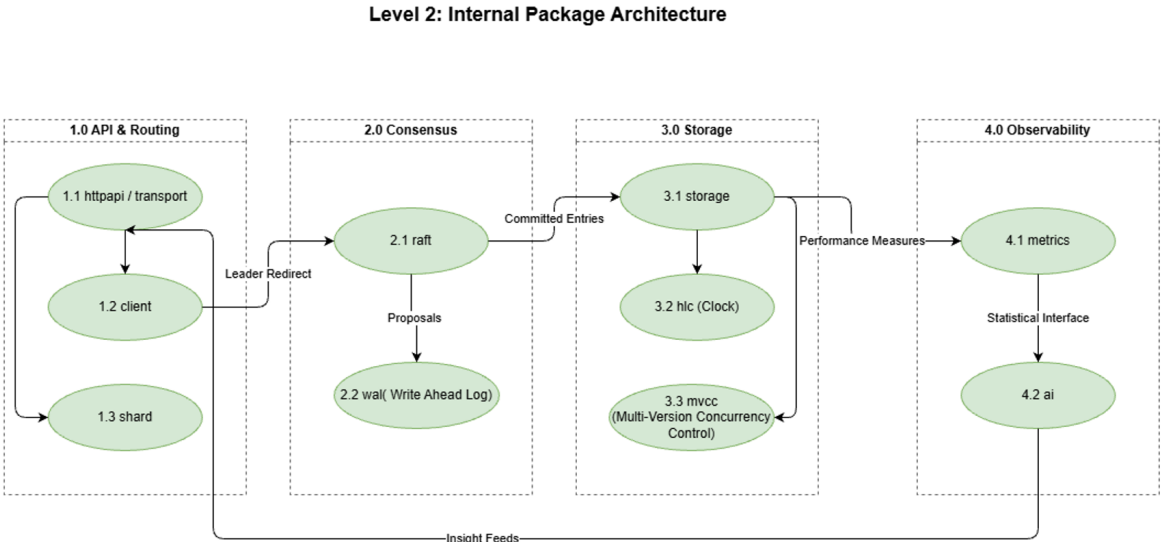


Figure 6.3: Level 2 internal package architecture. Original supplied image retained as a separate portrait-page figure.

1. **1.0 API & Routing:** receives and validates requests, identifies shard ownership, and forwards requests to the appropriate cluster node.
2. **2.0 Consensus:** receives proposals and coordinates replication and commit decisions through Raft.
3. **3.0 Storage:** receives committed entries, persists state, and maintains timestamped versions.
4. **4.0 Observability:** consumes metrics streams and emits alerts/insights for the external operator.

The data store D1 represents persistent storage. The Fsync Append flow highlights the durability boundary: log data must be safely persisted before the system relies on it for recovery and commitment.

6.4 Level 2 Internal Package Interpretation

The Level 2 diagram divides the system into four package groups. The package labels below preserve the names shown in the supplied diagram, including the implementation-level spelling where present.

Package group	Components shown	Responsibility and flow interpretation
1.0 API & Routing	1.1 `httpapi/transport`; 1.2 `client`; 1.3 `shard`	Transport accepts and encodes requests; the client library handles calls and leader hints; shard routing maps keys to the appropriate cluster/shard. The diagram shows client-to-Raft leader redirect and transport/shard interactions.
2.0 Consensus	2.1 `raft`; 2.2 `wal` (Write Ahead Log)	Raft handles leader election, proposals, replication, and commitment. The WAL stores records for durable recovery. Proposals flow to the WAL; committed entries flow from Raft toward storage.
3.0 Storage	3.1 `storage`; 3.2 `hlc` (Clock); 3.3 `mvcc` (Multi-Version Concurrency Control)	Storage applies committed operations; HLC supplies timestamp values; MVCC manages version chains, snapshot visibility, and conflict checks.
4.0 Observability	4.1 `metrics`; 4.2 `ai`	Metrics collects performance measurements and exposes a statistical interface. AI/operational intelligence consumes allowed statistics and produces insight feeds.

6.4.1 Named internal flows

- **Leader Redirect:** a request received by a non-leader can be answered with a leader hint so the client can retry against the current leader.
- **Proposals:** the client/transport path submits candidate operations to the Raft subsystem for ordered replication.

- **Committed Entries:** after the consensus commit condition is met, entries are applied by storage.
- **Fsync Append:** the write-ahead log persists records before the system relies on them for crash recovery.
- **Performance Measures:** storage/cluster activity contributes counters and latency measurements to the metrics component.
- **Statistical Interface:** metrics are made available to the operational intelligence component through a narrow interface rather than direct access to database internals.
- **Insight Feeds:** operational findings are surfaced to the API/dashboard path for the operator.

- `httpapi/transport`: protocol boundary and request/response handling.
- `client`: client-side calls, leader hint caching, and retry/redirect logic.
- `shard`: key-to-shard routing and shard-level dispatch.
- `raft`: leader election, log replication, and commit progression.
- `wal`: append-only durable records and replay support.
- `storage`: application of committed entries and durable state management.
- `hlc`: hybrid logical timestamps used by version and transaction logic.
- `mvcc`: version chains, snapshot reads, write-set validation, and conflict checks.
- `metrics`: counters, latency measurements, histograms, and metric exposition.
- `ai`: anomaly interpretation and bounded operational query assistance.

6.5 Balancing and Consistency Check

A balanced DFD preserves the external flows when moving from context to Level 1: client requests still enter the system and results still return to the client; operator-facing insights still originate from observability; and diagnostic evidence/hypotheses remain an external interaction. Level 2 decomposes internal subsystems without inventing a new external actor. Data stores are accessed through processes, not directly by external entities.

Chapter 7

UML Structural Design

7.1 Class Diagram

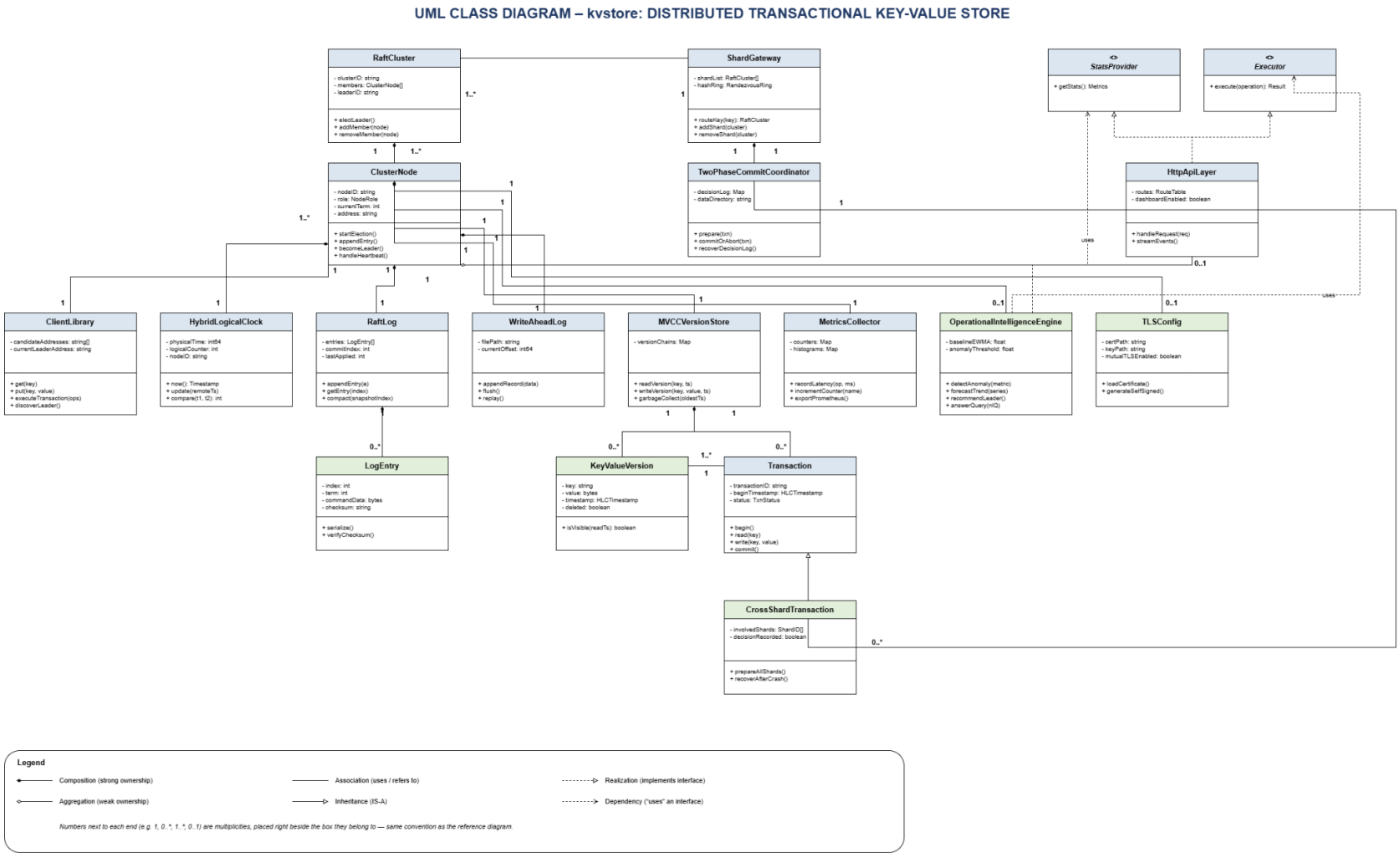


Figure 7.1: UML class diagram supplied for kvstore.

7.2 Important Classes and Responsibilities

Class / component	Responsibility
`RaftCluster`	Maintains cluster membership and leader identity; coordinates cluster-level operations.
`ClusterNode`	Represents a node participating in election, append, heartbeat, and leadership behaviour.
`ShardGateway`	Maps keys to shards and dispatches requests to the appropriate Raft cluster.
`TwoPhaseCommitCoordinator`	Coordinates prepare/commit/abort across shards and persists decisions for recovery.
`ClientLibrary`	Provides client-facing operations, transaction calls, and leader discovery.
`RaftLog`	Stores ordered log entries and tracks committed/applied positions.
`WriteAheadLog`	Appends records durably and replays records during recovery.
`HybridLogicalClock`	Generates and compares timestamps for ordering events and versions.
`MVCCVersionStore`	Maintains version chains and applies snapshot visibility/conflict rules.
`Transaction`	Represents transaction ID, begin timestamp, buffered reads/writes, and outcome.
`CrossShardTransaction`	Represents a transaction that spans multiple shards and its durable decision state.
`MetricsCollector`	Collects operation counts, latency and histogram measurements for monitoring.
`OperationalIntelligenceEngine`	Interprets supported operational queries and builds bounded diagnostic context.
`TLSConfig`	Loads certificate material and configures secure transport validation.
`HttpApiLayer`	Exposes HTTP routes and event streaming; delegates core work to internal components.

7.3 Design Principles Reflected

- **Separation of concerns:** transport, consensus, persistence, transactions, and metrics have different responsibilities.
- **Dependency direction:** the API layer delegates to internal services rather than implementing consensus or storage logic itself.
- **Encapsulation:** the WAL and MVCC store own their data structures and expose operations rather than exposing raw internals.
- **Explicit transaction state:** transaction and cross-shard transaction objects make outcome and recovery responsibilities visible.
- **Operational isolation:** metrics and AI diagnostics observe system behaviour without becoming commit authorities.

Interaction and Sequence Design

8.1 Scenario 1: PUT with Leader Redirect

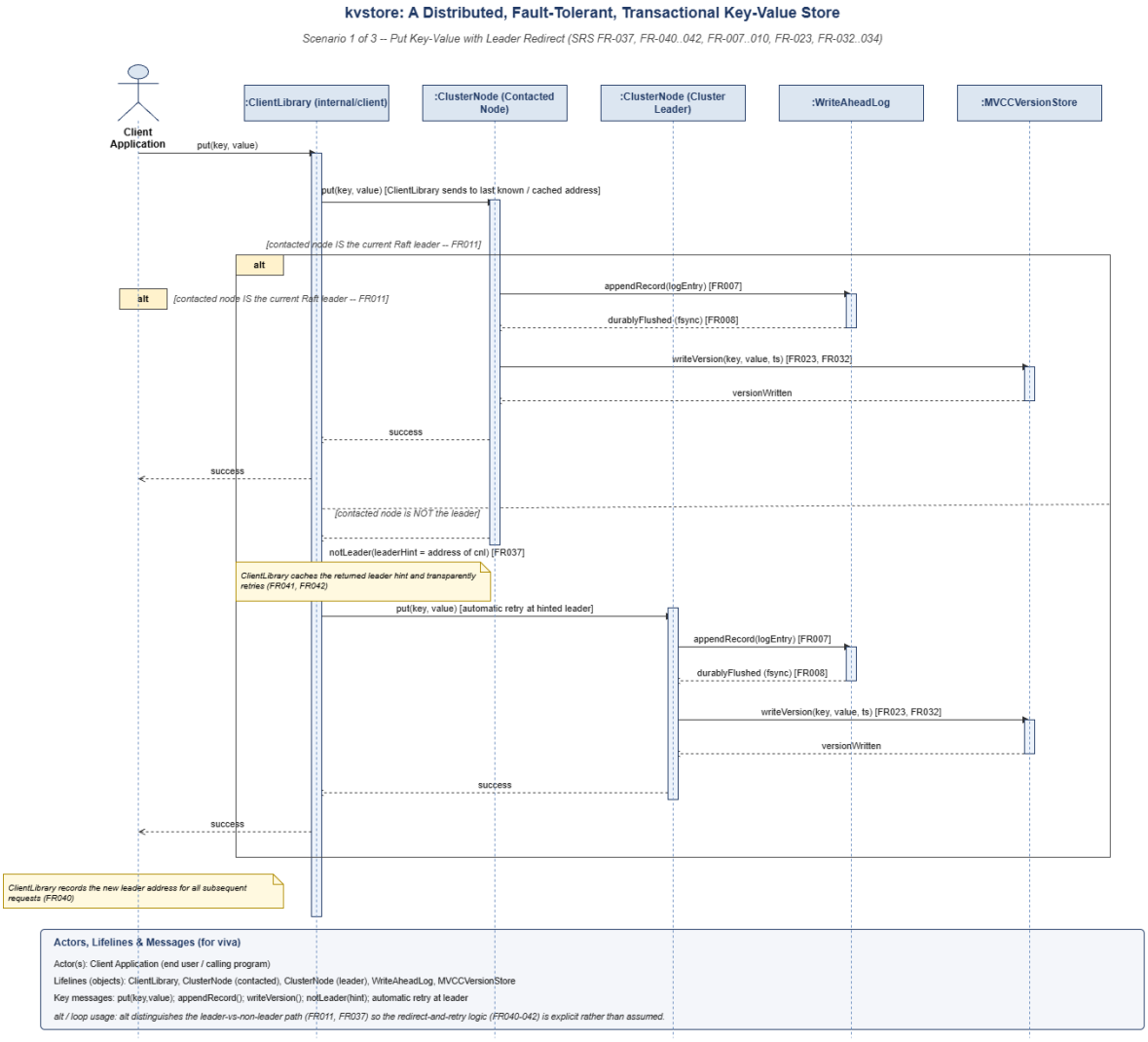


Figure 8.1: Sequence diagram for a PUT request when the contacted node may not be the leader.

Narrative: The client library sends a PUT request to the contacted node. If that node is the leader, it appends the log entry, ensures durable persistence, applies the version, and returns success. If it is a follower, it returns a leader hint; the client caches the hint and retries against the leader. This makes redirect-and-retry behaviour explicit rather than assuming every client initially contacts the leader.

8.2 Scenario 2: Transaction with Snapshot Isolation

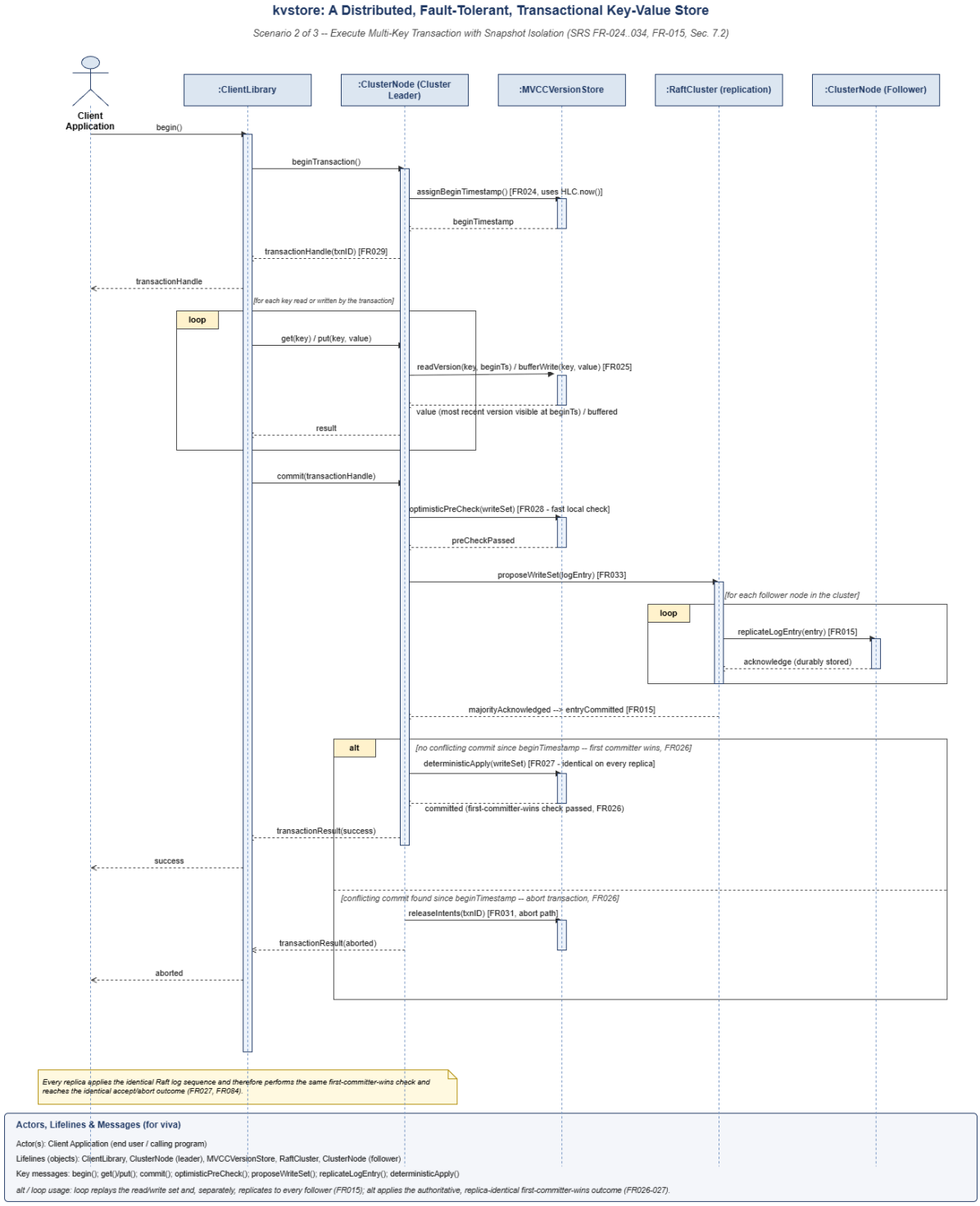


Figure 8.2: Sequence diagram for a transaction using snapshot visibility and first-committer-wins validation.

Narrative: A transaction begins and receives a begin timestamp. Reads use the snapshot visible at that timestamp; writes are buffered. At commit, the system performs a pre-check and proposes the write set through the replicated path. If no conflicting committed write invalidates the transaction, the transaction commits. Otherwise, the transaction is aborted and its intents are released. A conflict must not partially overwrite the winning transaction's state.

8.3 Scenario 3: Cross-Shard Two-Phase Commit

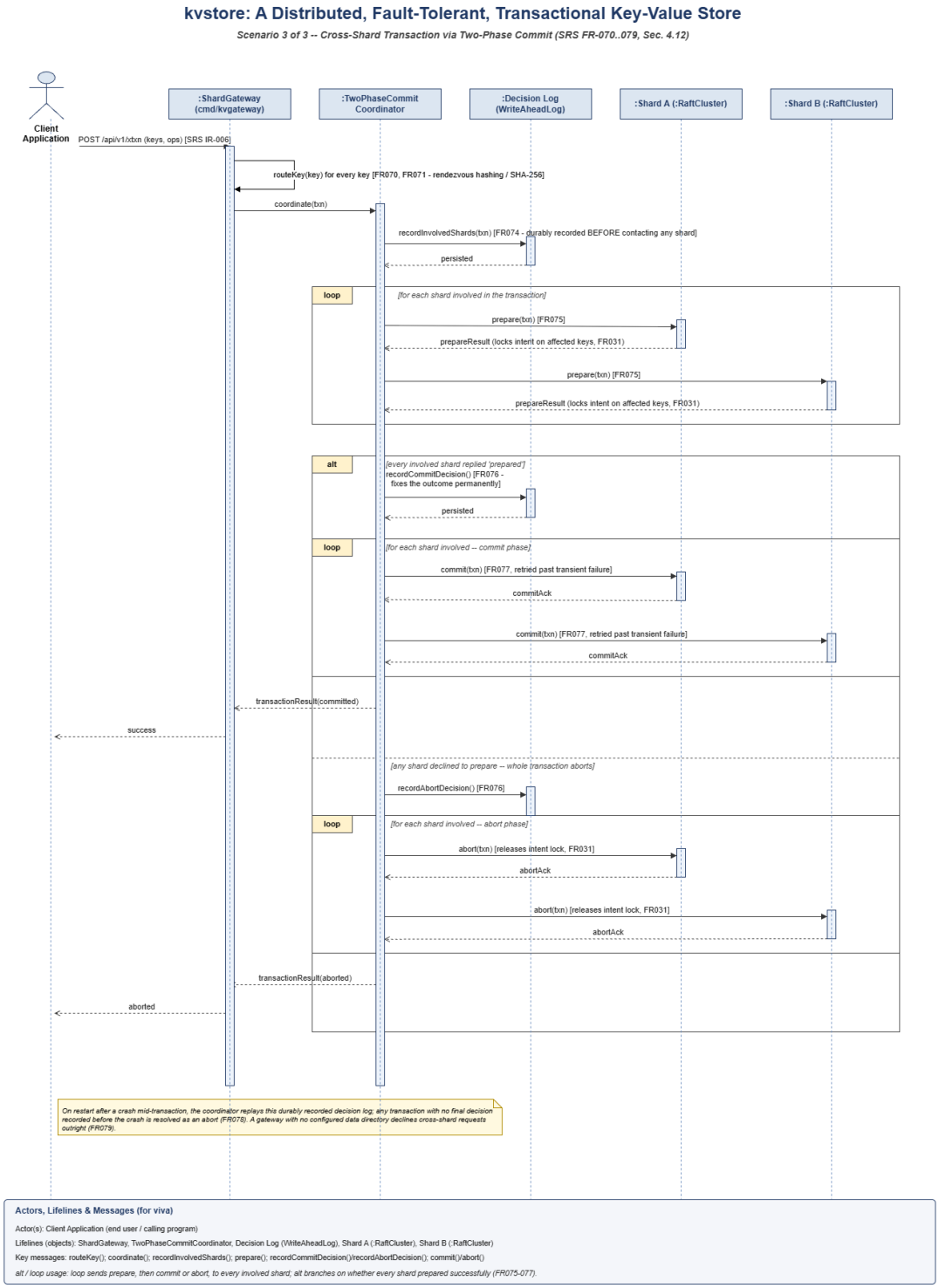


Figure 8.3: Sequence diagram for a transaction spanning two shards using two-phase commit.

Narrative: The gateway determines which shards participate and sends the transaction to the coordinator. The coordinator durably records the transaction's involved shards before contacting participants. Each shard prepares and holds the relevant intent locks. If all participants prepare, the coordinator records a durable commit decision and asks all participants to commit. If any participant declines, the coordinator records abort and releases participant intents. On restart, the coordinator consults the durable decision log to resolve an in-doubt transaction.

8.4 Cross-Scenario Invariants

1. A client receives success only after the system reaches the required commit point.
2. A follower redirect does not itself mutate committed state.
3. Snapshot reads are governed by the transaction's visibility timestamp.
4. A conflicting transaction must abort rather than overwrite a committed conflicting write.
5. A cross-shard transaction must not report a successful final commit if only a subset of participating shards committed.
6. A durable decision record is the recovery authority for an in-doubt cross-shard transaction.

Modular Design

9.1 Design Objectives

The modular design exercise focuses on high cohesion and low coupling. Each module should have one main reason to change, expose a small interface, and avoid depending on implementation details owned by another module.

9.2 Module Architecture

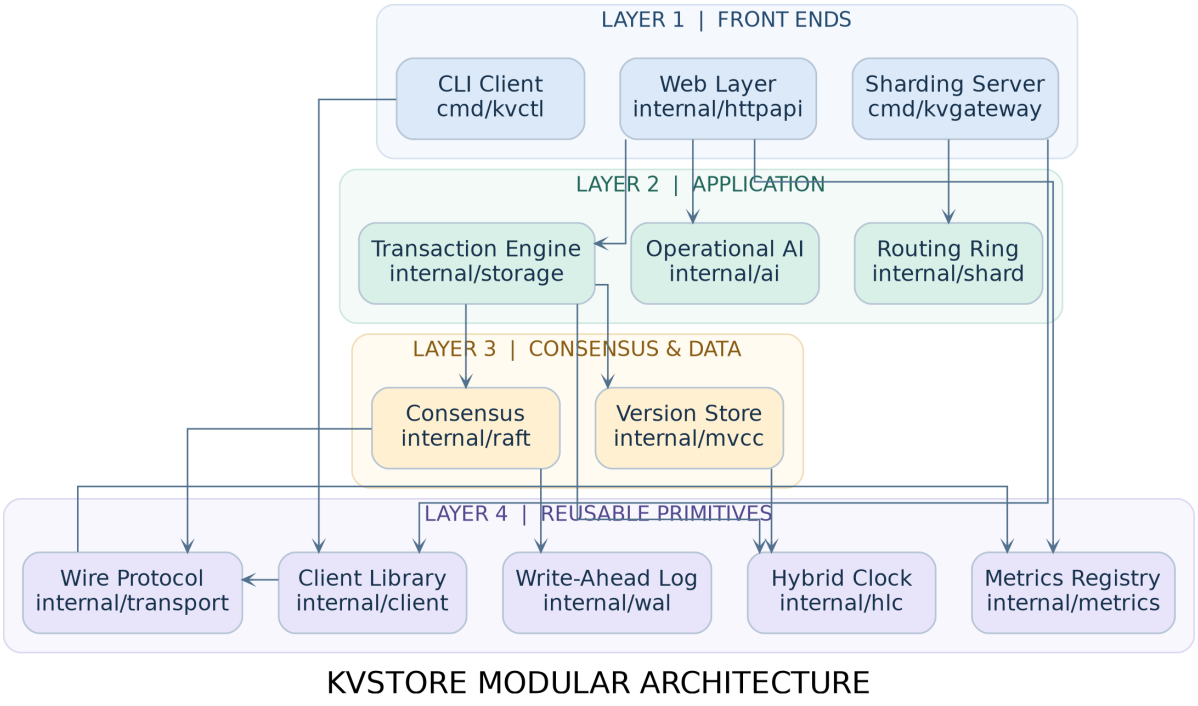


Figure 9.1: Module architecture for kvstore, based on the supplied Lab 8 artefact.

9.3 Module Responsibilities and Interfaces

Module	Responsibility	Interface / data exchanged
HTTP API / Transport	Accepts requests and serializes responses	Request objects, status codes, response payloads
Client Library	Convenient client operations and redirect handling	PUT/GET/DELETE, transaction handles, leader hints
Shard Router	Maps keys to shard ownership	Key, shard ID, route result
Raft / Consensus	Leader election and replicated log commitment	Proposals, log entries, acknowledgements, commit index
WAL	Durable append and replay	Log record, append result, replay stream
Storage	Applies committed operations	Committed command, key/value state
HLC	Produces ordered hybrid timestamps	Timestamp generation and comparison
MVCC	Version visibility and write conflict checks	Read timestamp, key/version, write set, commit/abort result
2PC Coordinator	Coordinates distributed transaction outcomes	Participant list, prepare response, durable decision
Metrics	Collects and exports operational measurements	Counters, histograms, scrape endpoint
Operational Intelligence	Parses bounded operational queries and forms diagnostic context	Query intent, metric evidence, hypothesis result
TLS Configuration	Loads and validates transport trust material	Certificate configuration and secure transport settings

9.4 Cohesion and Coupling Review

- **High cohesion:** WAL operations remain in the WAL module; version visibility and conflict rules remain in MVCC; metrics collection remains in the metrics module.
- **Low coupling:** modules communicate through request types and method contracts rather than reaching into one another's private state.
- **Replaceability:** the external LLM provider can be changed or unavailable without changing Raft commitment or storage correctness.
- **Testability:** modules can be unit-tested with controlled dependencies, while integration tests verify the contracts between modules.
- **Change isolation:** transport changes should not require changes to transaction conflict logic; metrics presentation changes should not modify the WAL format.

9.5 Architecture Decisions

Decision	Reason	Trade-off
Raft for replication	Provides a clear leader-based replicated-log model	Requires quorum for progress and careful leader-change handling
WAL for durability	Enables recovery by replaying durable records	Adds write-path I/O and log lifecycle work
MVCC for transaction reads	Supports snapshot reads and conflict checks	Requires version management and cleanup strategy
Two-phase commit across shards	Coordinates atomic outcome across shard participants	Can block while a durable decision is unresolved
Separate observability path	Protects correctness path from diagnostic dependencies	Requires careful context assembly and failure handling
LLM as advisory assistant	Can provide operator-facing hypotheses from metrics/evidence	Outputs may be wrong and must not be executed as authority

Implementation Overview

10.1 Technology and Repository View

The supplied project materials describe a Go implementation and a package-oriented repository. The exact runtime configuration may vary by build, so deployment values should be taken from the checked-out repository and environment configuration rather than inferred from this report.

Conceptual request path: Client → HTTP API / Client Library → Shard Router → Raft Leader → WAL / Replication → MVCC Storage → Client Result. Metrics and AI diagnostics observe selected signals alongside this path.

10.2 Key Implementation Areas

1. **Transport and API:** request decoding, validation, routing, status-code mapping, and response serialization.
2. **Leader discovery:** follower response with leader hint, client-side hint caching, and retry against the leader.
3. **Replicated commit:** proposal creation, WAL append, follower replication, quorum acknowledgement, commit advancement, and application.
4. **Durable recovery:** replay committed log records and recover transaction-coordinator decisions as appropriate.
5. **MVCC transactions:** begin timestamp, snapshot reads, buffered writes, conflict check, and commit/abort outcome.
6. **Cross-shard coordination:** participant discovery, durable decision log, prepare, commit/abort, and restart recovery.
7. **Operational interfaces:** metrics, live events/dashboard, safe operational query handling, TLS validation, and optional AI diagnostics.

10.3 Error Handling Principles

- Return structured and meaningful errors for invalid input, missing keys, unsupported commands, untrusted certificates, and unavailable dependencies.
- Distinguish retryable leader-routing failures from terminal validation or transaction-conflict outcomes.

- Avoid reporting a final commit outcome before the relevant durability and consensus conditions have been satisfied.
- Treat observability and external AI failures as diagnostic failures, not as permission to bypass database rules.

Functional Test Strategy and Verification

11.1 Testing Objectives

The testing strategy checks functional correctness, concurrency control, durability, routing, security, operational query safety, metrics, and cross-shard atomicity. Test cases should be repeatable, linked to requirement identifiers, and assessed against observable expected results.

11.2 Test Levels

Level	Purpose	Examples
Unit	Validate a module's local logic	Key validation, MVCC visibility, timestamp comparison, query parsing
Integration	Validate interactions between modules	HTTP-to-storage path, WAL replay, Raft replication, TLS configuration
End-to-end	Validate a user-visible workflow	PUT/GET/DELETE, leader redirect, transaction outcome, metrics query
Fault / recovery	Validate behaviour around failure and restart	Crash after committed write, coordinator restart, participant failure
Negative / security	Ensure unsafe or invalid actions are rejected	Absent key, untrusted TLS certificate, unsupported operational query

11.3 Test Register

Traceability note. The separate Lab 9 worksheet uses abbreviated labels such as FR-04/FR-05 for its test-design register. Those labels do not match the formal SRS namespace, where SRS FR 004 concerns hybrid logical clock behaviour. To avoid an identifier collision, this consolidated register uses formal SRS FR identifiers for the mapped test scenarios.

Test ID	Req. ID	Scenario / input	Type	Expected result
TC-01	SRS FR 032, 034, 066	PUT a new key 'course' with value 'MCA', then GET it	Positive	Value is stored and returned

Test ID	Req. ID	Scenario / input	Type	Expected result
TC-02	SRS FR 066	GET a key that does not exist	Negative	Not-found result is returned
TC-03	SRS FR 032, 066	DELETE an existing key, then GET it	Positive	Key is absent after deletion
TC-04	SRS FR 066	DELETE a key that is already absent	Negative	Defined absent-key response; no corruption
TC-05	SRS FR 023–028, 032–034	Commit a transaction with two writes to keys in the same shard	Positive	Both writes commit consistently
TC-06	SRS FR 026–028	Two transactions concurrently update the same version	Negative	Conflicting transaction aborts; winner is not overwritten
TC-07	SRS FR 007–010, 083	Restart a node after a committed write	Positive	Committed value is recovered
TC-08	SRS FR 037, 040–042	Send a client request to a follower	Positive	Leader hint/redirect is returned and retry can succeed
TC-09	SRS FR 015–018	Rejoin a follower after a committed write	Positive	Follower catches up to committed state
TC-10	SRS FR 043–048	Request Prometheus-style metrics	Positive	Metrics endpoint returns valid metrics payload
TC-11	SRS FR 058–062	Submit an unsupported natural-language operational command	Negative	Safe help/unsupported response; no arbitrary action
TC-12	SRS FR 038–039, 063–065	Connect using an untrusted TLS certificate	Negative	Connection is rejected in secure mode
TC-13	SRS FR 066	Send a valid REST PUT request	Positive	Appropriate success status and response body
TC-14	SRS FR 070–079	Submit a transaction whose keys map to different shards under a rejected/failed participant condition	Negative	No partial commit; final outcome is aborted/rejected
TC-15	SRS FR 074–078	Restart coordinator after a durable cross-shard decision	Positive	Recovery follows the recorded decision
TC-16	SRS FR 059–062	Make the LLM provider unavailable during an operational query	Negative / resilience	Core data operations remain correct; diagnostic failure is surfaced

11.4 Detailed Test Procedure: PUT and GET

Preconditions: a healthy cluster is running and the client can reach the API.
Steps:

1. Send a valid PUT for key ‘course‘ and value ‘MCA‘.
2. Confirm the response indicates successful completion.

3. Send GET for key ‘course‘.
4. Compare the returned value with ‘MCA‘.
5. Repeat with an absent key to validate not-found behaviour.

Pass rule: the successful PUT is followed by a GET returning the stored value; absent keys return the documented not-found response without creating data.

11.5 Detailed Test Procedure: MVCC Conflict

Preconditions: two concurrent transactions can read/write the same key/version.
Steps:

1. Begin transactions T1 and T2 before either commits.
2. Have both transactions read the same key/version.
3. Let T1 write and commit first.
4. Attempt to commit T2 with a conflicting write.
5. Read the key and inspect both transaction outcomes.

Pass rule: the first valid commit remains visible; the conflicting transaction is aborted or rejected; no partial conflicting write overwrites the winner.

11.6 Detailed Test Procedure: Crash and Restart Recovery

Preconditions: a test cluster with persistent data directory and a controllable node process.
Steps:

1. Commit a known key–value write and confirm success.
2. Stop or terminate the node after the commit response.
3. Restart the node using the same persistent data directory.
4. Wait for recovery and cluster readiness.
5. Read the key through the API and compare the value.

Pass rule: the committed value is available after restart; no uncommitted partial state is presented as committed.

11.7 Detailed Test Procedure: Cross-Shard Atomicity

Preconditions: two or more shards and a test transaction whose keys route to different shards.
Steps:

1. Submit a transaction with writes to keys assigned to at least two shards.
2. Force a participant prepare failure or a controlled coordinator interruption in a test environment.
3. Inspect the returned transaction outcome and the values on all involved shards.
4. Restart the coordinator if needed and allow recovery to finish.

Pass rule: the transaction resolves according to its durable decision; a failed prepare does not result in only a subset of the intended writes being committed.

11.8 Traceability Summary

Requirement group	Representative IDs	Verification evidence
Key–value API	SRS FR 032–034, 066	TC-01 through TC-04 and TC-13; HTTP route tests
Replication / durability	SRS FR 007–022, 083–084	TC-07 through TC-09; WAL, Raft, and restart tests
Transactions / MVCC	SRS FR 023–034	TC-05, TC-06; snapshot and conflict tests
Observability / safe query	SRS FR 043–062, 067–069	TC-10, TC-11, TC-16; metrics and query-engine tests
TLS / security	SRS FR 038–039, 063–065	TC-12; certificate trust test
Cross-shard transaction	SRS FR 070–079	TC-14, TC-15; coordinator and gateway tests

11.9 Implementation Evidence and Status Discipline

The supplied repository evidence identifies automated tests in the areas of REST key operations and metrics export, safe natural-language query handling, TLS trust validation, MVCC conflict behaviour, crash/restart durability, cross-shard atomicity, and gateway rejection of unsupported cross-shard cases. The project materials also state that a prior full test suite passed. A fresh rerun was previously blocked in the available environment because the required Go 1.25.0 toolchain download could not be completed.

Important interpretation: source code and existing test definitions demonstrate that checks are implemented in the repository, but do not prove that every test was freshly executed against the exact submitted build. For a formal release record, capture the commit hash, Go version, command, timestamp, and test output. The report therefore describes expected outcomes and known test coverage without fabricating a new pass result.

Security, Reliability, and Operational Considerations

12.1 Security

- Validate TLS certificates in secure mode and reject untrusted server certificates.
- Keep administrative operations behind controlled interfaces and avoid exposing privileged actions through free-form natural-language commands.
- Bound the diagnostic evidence sent to external AI providers and avoid sending secrets or unnecessary data.
- Treat LLM responses as untrusted suggestions; they must not directly mutate cluster state or override transaction outcomes.

12.2 Reliability and Recovery

- Use the WAL and Raft commit position as the basis for committed-state recovery.
- Recover transaction coordinator state from durable decision records.
- Ensure replicas apply committed entries deterministically.
- Handle transient transport failures through bounded retries and explicit error responses.
- Keep the core data path operational when the observability or external LLM path is unavailable, subject to deployment configuration.

12.3 Operational Monitoring

The operator should be able to inspect request counts, latency distributions, error rates, cluster membership, leader state, replication progress, and transaction outcomes where the implementation exposes them. Metrics names and labels should be taken from the running build's metrics endpoint. AI-generated hypotheses should be displayed alongside the evidence and should be labelled as hypotheses, not confirmed causes.

Chapter 13

Limitations and Future Enhancements

13.1 Current Documentation Limitations

- Performance and scale claims are not stated as measured results because no reproducible benchmark report was included as a source for this consolidated document.
- Production-grade operational targets, backup/restore policy, data-retention policy, and deployment-specific capacity limits require explicit values.
- Exact API schemas and command-line flags should be checked against the repository version used for the final demonstration.
- Some design artefacts use conceptual package labels; the concrete source tree should be used to confirm the current implementation names before a production release.

13.2 Recommended Enhancements

1. Add a reproducible benchmark suite for throughput, tail latency, and cluster scaling.
2. Add fault-injection automation for leader changes, disk errors, network partitions, and coordinator crashes.
3. Add explicit operational runbooks for backup, restore, certificate rotation, and incident response.
4. Add a versioned OpenAPI specification and generate client examples from the same source.
5. Add structured audit events for operator actions and AI diagnostic requests.
6. Add retention and cleanup policies for WAL segments and MVCC versions, with tests for safe compaction.
7. Capture CI artefacts including test reports, race detector results, coverage summaries, and build provenance.

Chapter 14

Conclusion

This report presents *kvstore* as a coherent software product rather than a collection of isolated lab submissions. The SRS defines the expected behaviours; the selected iterative, risk-driven process gives a way to build and validate them incrementally; the use-case and DFD views define system boundaries and information flows; UML diagrams explain structure and runtime interactions; modular design assigns responsibilities to focused packages; and the functional test strategy links requirements to observable results.

The central correctness principles are durable replicated commitment, deterministic application, MVCC conflict detection, atomic cross-shard outcomes, safe operational interfaces, and strict separation between database authority and AI-assisted diagnosis. The next step for a release-ready submission is to run the full test suite against the exact submitted revision and attach reproducible execution evidence.

Consolidation Map: Laboratory Artefacts

Lab / tutorial	Artefact incorporated	Where represented in this report
Lab 1	Software Requirements Specification	Chapter 3: functional and non-functional requirements; Appendix A: formal SRS FR index
Lab 2	Process-model selection	Chapter 4: process model, development increments, and risk register
Lab 3	Use-case diagram	Chapter 5: actors and principal use cases
Lab 4	Context, Level 1 and Level 2 DFDs	Chapters 2 and 6: system context, subsystem flows, and package decomposition
Tutorial 5	UML class diagram	Chapter 7: class responsibilities and structural principles
Tutorial 6	Three sequence diagrams	Chapter 8: leader redirect, snapshot isolation, and cross-shard 2PC
Lab 7	No separate Lab 7 source artefact was identifiable in the files supplied for this consolidation	Related architecture, implementation, and verification topics are covered in Chapters 9–12. If Lab 7 has a specific handout or required deliverable, it should be added rather than inferred.
Lab 8	Modular design report and module architecture	Chapter 9: module responsibilities, interfaces, cohesion, and coupling
Lab 9	Functional test design and student submission	Chapter 11: test strategy, test register, detailed procedures, traceability, and evidence status

Chapter B

Expanded Lab 8 Modular-Design Record

This appendix retains the important project-specific detail from the original Lab 8 modular-design submission instead of reducing it to a one-page summary.

B.1 Project Requirements Used for Modularisation

ID	Requirement	Meaning for module design
R1	Key–value operations	Put, Get, Delete, and atomic multi-key transactions.
R2	Fault-tolerant replication	Raft replication across machines; the cluster should continue when a minority of nodes fail.
R3	Strong isolation	Snapshot isolation and first-committer-wins conflict detection, applied consistently on replicas.
R4	Durability	Accepted changes are written to a checksummed WAL and fsynced before acknowledgement.
R5	Deterministic ordering	HLC timestamps provide a total order that nodes do not reverse.
R6	Observability	Prometheus/JSON metrics, REST API, and a live browser dashboard.
R7	Operational intelligence	Statistical anomaly detection, correlation, forecasting, leadership advice, auto-tuning, and natural-language queries.
R8	Horizontal scaling	Independent shard clusters behind a gateway, with supported cross-shard two-phase-commit transactions.

The original design states that kvstore uses Go's standard library without third-party libraries; Raft, MVCC, WAL, wire protocol, and HTTP functionality are implemented in the project. This constraint makes small, well-defined package responsibilities especially important.

B.2 Responsibility Inventory

The 25 responsibilities recorded in the original modular-design exercise are: (1) store a value under a key; (2) read a value; (3) delete a key; (4) execute several operations as one atomic transaction; (5) detect write-write conflicts; (6) replicate operations; (7) elect a leader and recover after leader failure; (8) keep the log safe on disk before acknowledgement; (9) generate ordered timestamps; (10) recover state after a crash through replay and snapshots; (11) serve requests over the network; (12) redirect

clients that reach a non-leader; (13) measure latency, throughput, and failures; (14) export metrics to Prometheus and JSON; (15) show a live dashboard; (16) detect anomalies; (17) correlate and forecast metric trends; (18) advise leadership transfers; (19) auto-tune GC and snapshot parameters; (20) answer plain-English cluster questions; (21) route keys to shards; (22) run supported transactions across shards; (23) discover the current leader and retry there; (24) optionally encrypt connections with TLS/mTLS; and (25) inject partitions, kills, and delays in tests.

B.3 Thirteen Modules and Package Mapping

Module	Package	Primary responsibility
CLI Client	`cmd/kvctl`	Command-line interface for reading, writing, deleting, and checking node status through the client library.
Web Layer	`internal/httpapi`	REST API, Server-Sent Events, and embedded browser dashboard.
Sharding Server	`cmd/kvgateway`	Single logical gateway in front of independent shard clusters.
Transaction Engine	`internal/storage`	Raft state machine and Begin/Get/Put/Delete/Rollback/Commit transaction operations.
Operational AI	`internal/ai`	Metric analysis, anomaly detection, correlation, forecasting, advice, auto-tuning, and natural-language queries.
Routing Ring	`internal/shard`	Deterministic key-to-shard routing using rendezvous hashing.
Consensus	`internal/raft`	Elections, replication, snapshots, membership, and ordered commit.
Version Store	`internal/mvcc`	Version chains, snapshot isolation, and conflict checks.
Hybrid Clock	`internal/hlc`	Hybrid logical timestamps and ordering.
Write-Ahead Log	`internal/wal`	Checksummed append, fsync, and replay.
Wire Protocol	`internal/transport`	Envelope encoding, request/reply transport, and leader hints.
Metrics Registry	`internal/metrics`	Counters, gauges, histograms, and metric exposition.
Fault Injection	`internal/fault`	Test-only simulated partitions, process kills, and delays.

B.4 Module Interfaces

B.4.1 Interface 1: Transaction API

The storage engine exposes a transaction handle and operations. The API listed in the original lab is:

```
func (e *Engine) Begin(ctx context.Context) (*Txn, error)
func (t *Txn) Get(key string) (value []byte, ok bool)
func (t *Txn) Put(key string, value []byte)
```

```
func (t *Txn) Delete(key string)
func (t *Txn) Rollback()
func (t *Txn) Commit(ctx context.Context) (hlc.Timestamp, error)
```

Call	Input	Output / participating modules
Begin	Context	Transaction handle with snapshot timestamp; Web Layer → Transaction Engine.
Get	Key	Value bytes or not-found; Web Layer → Transaction Engine.
Put/Delete	Key and optional value	Change buffered in transaction; Web Layer → Transaction Engine.
Commit	Open transaction	Commit timestamp or conflict abort; Transaction Engine → Consensus and Version Store.
Rollback	Open transaction	Transaction changes discarded; Web Layer → Transaction Engine.

B.4.2 Interface 2: Wire Envelope

The transport envelope distinguishes Raft traffic (RequestVote, AppendEntries, InstallSnapshot, TimeoutNow) from client requests (Get, Put, Delete, transaction). The reply carries an error string and can include `NotLeader=true` plus `LeaderHint` so a client can retry. One request uses a fresh TCP connection and receives one envelope reply before the connection closes, according to the lab interface description.

B.4.3 Interface 3: StatsProvider

The operational AI is deliberately limited to a small statistics interface implemented by an adapter in the HTTP layer:

```
type StatsProvider interface {
    MetricsSnapshot() metrics.Snapshot
    NodeObservations() []NodeObservation
    VersionCount() int
}
```

Metrics snapshots provide counters, gauges, and histograms; node observations provide latency, errors, replication lag, and queue depth; version count provides a signal for MVCC garbage-collection pressure. The interface prevents the AI package from directly depending on Raft or storage internals.

B.4.4 Interface 4: Key Routing

```
func NewRing(shards []ID) *Ring
func (r *Ring) ShardFor(key string) ID
```

The routing contract is deterministic: the same key and same shard set return the same shard on each machine and after restart. The lab specifies that cross-shard requests must not be silently applied non-atomically; they are either handled through the dedicated coordinator path or rejected clearly where that path is not configured.

B.5 Cohesion and Coupling Decisions

C1 — WAL as a generic append-only log. The WAL accepts arbitrary byte records, appends and checksums them, fsyncs them, and replays them. It does not parse key/value semantics, transaction meaning, or Raft commands. Combining those domain responsibilities with log mechanics was rejected because it would create unrelated reasons to change and would make independent testing harder.

C2 — Operational AI as a statistics consumer. The AI package contains analysis/query logic but does not own network transport, persistence, or storage operations. Its narrow interface is the boundary between operational insights and the core database.

D1 — Keep HLC independent of storage. The clock returns a timestamp type; it does not import MVCC or storage. This keeps a fundamental ordering primitive stable and independently testable.

D2 — Keep shard routing deterministic and stateless. The gateway asks the routing ring for the shard responsible for a key. It should not duplicate database transaction logic or depend on internal storage structures.

B.6 Coupling Numericals from Lab 8

The original module graph counts 14 directed production dependencies across 13 drawn modules. Average coupling is therefore:

$$\text{Average coupling} = \frac{14 \text{ connections}}{13 \text{ modules}} \approx 1.08.$$

This is an average graph connection count, not a universal quality score. The graph is sparse and layered; the Fault-Injection module is test-only and its arrow is omitted from the production diagram.

The lab defines module instability as

$$I = \frac{F_{out}}{F_{in} + F_{out}},$$

where F_{in} is the number of modules depending on the module and F_{out} is the number of modules it depends on. $I = 0$ represents a stable primitive and $I = 1$ a volatile top layer. Examples from the lab:

- **Wire Protocol:** $F_{in} = 2$, $F_{out} = 1$, so $I = 1/(2 + 1) \approx 0.33$. Two modules rely on the envelope format, so the format should be extended compatibly rather than casually reshaped.
- **Transaction Engine:** $F_{in} = 1$, $F_{out} = 3$, so $I = 3/(1 + 3) = 0.75$. It orchestrates lower-level modules but few modules depend on its internals, so its high-level API can evolve with less downstream impact.

Module	F_{in}	F_{out}	I	**Interpretation recorded in Lab 8**
CLI Client	0	1	1.00	Top of stack; most volatile.
Sharding Server	0	2	1.00	Top-level gateway.
Web Layer	0	3	1.00	Composition root by design.
Transaction Engine	1	3	0.75	High-level orchestrator.
Consensus	1	2	0.67	Middle layer.
Version Store	1	1	0.50	Balanced.
Client Library	2	1	0.33	Fairly stable.
Wire Protocol	2	1	0.33	Fairly stable.
Hybrid Clock	2	0	0.00	Stable primitive.

Module	F_{in}	F_{out}	I	Interpretation recorded in Lab 8
Metrics Registry	2	0	0.00	Stable primitive.
Write-Ahead Log	1	0	0.00	Stable primitive.
Operational AI	1	0	0.00	Stable server-side leaf behind an interface.
Routing Ring	1	0	0.00	Stable server-side leaf.

The Lab 8 result states that the system decomposes into 13 modules in four layers, with 14 production dependencies and average coupling 1.08. The dependency gradient from volatile user-facing layers toward stable primitives is the quantitative argument for high cohesion and limited coupling in this particular graph.

B.7 Module-to-Code Traceability from Lab 8

Diagram node	Code location cited in the lab	Evidence / responsibility cited
Transaction Engine	`internal/storage/engine.go`	Begin/Get/Put/Delete/Rollback/Commit operations.
Operational AI	`internal/ai/engine.go`	Declares the StatsProvider interface.
Routing Ring	`internal/shard/ring.go`	ShardFor with SHA-256 rendezvous scoring.
Wire Protocol	`internal/transport/protocol.go`	Envelope kinds, NotLeader, and LeaderHint.
Consensus	`internal/raft/`	State-machine apply interface, transfer, and TimeoutNow.
Web Layer	`internal/httpapi/`	Adapter implements StatsProvider; REST, SSE, and dashboard.

Expanded Lab 9 Test Register and Execution Detail

This appendix preserves the test-design terminology and requirement IDs used in the original Lab 9 artefact. The Lab 9 shorthand FR-02, FR-03, etc. is a local test-assignment namespace and is not the same numbering scheme as the formal SRS IDs labelled SRS FR 001–084 earlier in this report. They are intentionally kept separate to avoid false one-to-one mappings.

C.1 Selected Lab 9 Requirements

Req. ID	Requirement under test	Test focus	Observable evidence
FR-02	A healthy cluster elects a valid leader per term	Leader availability	Node status/election log identifies one leader for the term.
FR-03	Leader replicates committed operations to followers	Replication	A committed value is visible after follower restart/rejoin.
FR-04	System returns a key value and found/not-found state	Read behavior	Existing key returns its value; missing key returns not-found.
FR-05	System creates or updates a key	Write behavior	A subsequent read returns the written value.
FR-06	System deletes a key	Delete behavior	Subsequent read reports the key as not found.
FR-07	Multiple KV operations execute in one-shard transaction	Atomic transaction	Commit result and final values match the transaction.
FR-08	MVCC uses snapshot isolation and first-committer-wins conflict detection	Conflicting writes	One conflicting commit is rejected; no unsafe partial result.
FR-10	Committed state is recoverable from CRC-checked WAL records	Durability	Committed value survives node restart.
FR-12	Client discovers leader and retries when initial node is not leader	Redirect / retry	Request through follower eventually receives correct result.

Req. ID	Requirement under test	Test focus	Observable evidence
FR-16	Metrics are rendered in Prometheus text format	Metrics export	Response contains expected metric names and valid text format.
FR-21	LLM actions are restricted to rule-engine-whitelisted operations	Operational query safety	Unsupported command is rejected without executing it.
FR-22	TCP optionally supports TLS/mTLS when configured	Certificate validation	Valid certificate connects; untrusted certificate is rejected.
FR-23	System exposes JSON endpoints for KV, transactions, status, metrics, and AI	REST API	Correct status code, content type, and JSON body.
FR-28	Gateway rejects transactions whose operations map to multiple shards	Cross-shard guard	Request is rejected before forwarding; no partial mutation.

C.2 Original Register A1: Scenario and Classification

Test ID	Req. ID	Scenario / input	Type	Status in source register
TC-01	FR-05, FR-04	PUT a new key `course` with value `MCA`.	Positive	Implemented
TC-02	FR-04	GET a key that does not exist.	Negative	Implemented
TC-03	FR-06, FR-04	DELETE an existing key, then GET it.	Positive	Implemented
TC-04	FR-06	DELETE a key that is already absent.	Negative	Implemented
TC-05	FR-07	Commit a transaction containing two writes to keys in the same shard.	Positive	Implemented
TC-06	FR-08	Two transactions concurrently update the same version.	Negative	Implemented
TC-07	FR-10	Restart a node after a committed write.	Positive	Implemented
TC-08	FR-12, FR-02	Send a client request to a follower.	Positive	Implemented
TC-09	FR-03	Rejoin a follower after a committed write.	Positive	Implemented
TC-10	FR-16	Request Prometheus metrics.	Positive	Implemented
TC-11	FR-21	Submit an unrecognized natural-language query.	Negative	Implemented
TC-12	FR-22	Connect using an untrusted TLS certificate.	Negative	Implemented

Test ID	Req. ID	Scenario / input	Type	Status in source register
TC-13	FR-23	Send a valid REST PUT request.	Positive	Implemented
TC-14	FR-73–FR-78	Submit an atomic cross-shard transaction through the dedicated endpoint.	Positive	Implemented
TC-15	FR-72	Submit a cross-shard request through the ordinary transaction endpoint.	Negative	Implemented

Status interpretation: "Implemented" is the source register's implementation/coverage label. It does not mean each manual procedure was freshly run while preparing this consolidated report. The Lab 9 source itself notes that the repository's full suite had passed previously, while a fresh rerun was blocked because the Go 1.25.0 toolchain was unavailable and downloading it was blocked in that environment.

C.3 Test Procedure Details and Pass Rules

C.3.1 TC-01: Create a key and read it back

Precondition: healthy cluster and a client endpoint are available. **Input:** key `course`, value `MCA`. Send PUT, check the success response, send GET for the same key, and compare the returned value. **Pass rule:** GET returns exactly `MCA` after successful PUT.

C.3.2 TC-06: Conflicting MVCC writes

Begin two transactions before either commits. Have both read the same key/version, let T1 write and commit, then attempt T2's conflicting commit. Inspect the outcomes and final value. **Pass rule:** the first valid commit remains visible; the losing conflicting transaction is aborted/rejected and does not overwrite the winner or apply a partial conflicting write.

C.3.3 TC-07: Crash and restart recovery

Commit a known write, stop the node after the commit response, restart it with the same persistent data directory, wait for readiness, then read the key. **Pass rule:** the committed value is recovered; incomplete or uncommitted state is not presented as committed.

C.3.4 TC-08: Request through a follower

Send a client operation to a follower rather than the known leader. Check that the reply identifies the non-leader condition and supplies a leader hint when known; allow the client library to retry. **Pass rule:** the operation reaches the leader and returns the correct result without applying the write twice.

C.3.5 TC-10: Prometheus metrics

Request the metrics endpoint and inspect the response content type and payload. **Pass rule:** the response is valid Prometheus text and includes the expected metric names/values for the running test process.

C.3.6 TC-11: Unsupported operational query

Submit a natural-language request that is not in the allowed command set. **Pass rule:** the system returns a help/unsupported response and does not execute an arbitrary operation.

C.3.7 TC-12: Untrusted TLS certificate

Enable the configured secure mode and connect using a certificate not trusted by the configured authority. **Pass rule:** the handshake/connection is rejected; the test must not disable verification to force a success.

C.3.8 TC-14 and TC-15: Cross-shard transaction boundary

For TC-14, use the dedicated cross-shard endpoint with operations routed to distinct shards and inspect the coordinator outcome and values on all participants. For TC-15, submit the same kind of cross-shard request to the ordinary transaction endpoint. **Pass rules:** the dedicated endpoint follows the supported atomic protocol; the ordinary endpoint rejects unsupported multi-shard operations before partial mutation.

C.4 Repository Test Evidence Named in Lab 9

The source archive and report name automated tests that correspond to the following areas. These names are evidence of repository test coverage; the report does not claim a new run unless a fresh execution log is available.

- REST key operations and metrics export: `TestHTTPPKVRoundTrip`, `TestHTTPMetricsPromAndJSON`.
- Safe natural-language query handling: `TestNLQueryEngineUnrecognizedFallsBackToHelp`.
- TLS trust validation: `TestClientRejectsUntrustedServerWithoutInsecureSkip`.
- MVCC conflict and durable restart recovery: `TestSnapshotIsolationWriteConflictFirstCommitterWins`, `TestCrashAndRestartPreservesCommittedWrites`.
- Cross-shard atomic commit and ordinary-endpoint rejection: `TestCrossShardTxnCommitsAtomicallyOnAllSha` `TestGatewayRejectsCrossShardTxn`.

C.5 Traceability and Execution Record

Test group	Requirement coverage	Evidence to retain for final execution record
CRUD and REST	FR-04–FR-06, FR-23	Request/response payloads, status code, final value, and test output.
Transactions and MVCC	FR-07, FR-08	Transaction IDs, commit/abort results, final values, and concurrency test output.
Durability and replication	FR-03, FR-10	Restart logs, recovered value, follower catch-up output.
Leader redirect	FR-02, FR-12	Initial follower address, leader hint, retry result.

Test group	Requirement coverage	Evidence to retain for final execution record
Metrics and operational query	FR-16, FR-21	Metrics response and safe fallback response.
TLS and gateway boundary	FR-22, FR-28, FR-72–FR-78	TLS handshake result; coordinator/participant results; proof of no partial mutation.

A formal execution record should include the exact source revision/commit, Go toolchain version, command executed, execution timestamp, pass/fail/blocked status, and saved output. Where a test has only been designed or its implementation has been identified, its status should not be changed to "Pass" without that evidence.

Quick Reference: Key Terms

Term	Meaning in kvstore
Raft	A consensus protocol used to elect a leader and replicate an ordered log.
WAL (Write-Ahead Log)	Durable record of operations used for recovery and replicated state application.
HLC (Hybrid Logical Clock)	Timestamp mechanism combining logical ordering with physical time information.
MVCC	Multi-version concurrency control; maintains versions and supports snapshot reads/conflict validation.
Snapshot isolation	Transaction reads from a consistent view associated with its begin timestamp.
First-committer-wins	When transactions conflict, a transaction that loses the commit validation is aborted rather than overwriting the winner.
Shard	A partition of key space; each key routes to its owning shard.
Two-phase commit	Coordinator protocol with prepare and commit/abort phases for a transaction involving multiple shards.
Leader redirect	A follower response that tells the client where to retry a leader-only operation.
Observability	Metrics, events, and diagnostics that help operators understand system behaviour.

Team and Submission Information

Role	Name	Enrollment ID
Team Leader	Neha Nupur	20251651064
Member	Aditi Garg	20251651008
Member	Shivam Kumar Soni	20251651084